

Security Audit of a Visa Consultancy Website Using Burp Suite

Aim

To examine the security of a Visa Consultancy website using Burp Suite Community Edition and understand how the website handles user requests, responses, and secure communication.

Objectives

- To understand the flow of HTTP and HTTPS communication.
- To inspect requests and responses exchanged between the browser and the website.
- To analyze how user information is transmitted through forms.
- To identify security mechanisms implemented by the website.
- To document observations related to website security and authentication.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Capturing and analyzing web traffic
Mozilla Firefox / Burp Browser	Accessing the target website
Visa Consultancy Website	Security testing target
SSL/TLS Protocol	Secure data transmission

Target Website

Website: <https://veerconsultancyglobal.com>

The selected website represents a visa consultancy portal and was suitable for performing basic web security analysis as required during the internship.

Step 1: Installing Burp Suite

Burp Suite Community Edition was downloaded from the PortSwigger website and installed successfully. After launching the application, a temporary project was created using the default settings.

Purpose

Burp Suite works as an intercepting proxy that allows monitoring and analyzing communication between the browser and the web server.

Step 2: Browser Proxy Configuration

Firefox was configured to send all traffic through Burp Suite by setting the proxy address to **127.0.0.1** and port **8080**. The option to use the proxy for all protocols was enabled.

Purpose

This setup allowed Burp Suite to capture incoming and outgoing web traffic for analysis.

Step 3: Installing the Burp Certificate

To inspect encrypted HTTPS traffic, the Burp Suite CA certificate was downloaded and imported into Firefox. The certificate was marked as trusted for website identification.

Purpose

Installing the certificate enabled the browser to communicate securely through Burp Suite without certificate warnings.

Step 4: Launching Burp Browser

Burp Suite's built-in Chromium browser was used for testing. The browser opened successfully without requiring any additional proxy configuration.

Advantages

- Automatic proxy setup
- HTTPS support enabled by default
- Easier traffic monitoring and interception

Step 5: Enabling Request Interception

The Intercept feature in Burp Suite was turned on through the Proxy tab. This allowed requests to be captured before reaching the server.

Step 6: Accessing the Website

The target website was opened using Burp Browser.

URL: <https://veerconsultancyglobal.com>

Observation

The website loaded successfully and established a secure HTTPS connection.

Step 7: Locating an Input Form

Since a traditional login page was not available, the website's contact and inquiry form was selected for testing. These forms accept user input and can be analyzed similarly to login forms.

Purpose

To study how user-provided information is processed and transmitted to the server.

Step 8: Submitting Test Data

Sample information was entered into the inquiry form to generate traffic for analysis.

Field	Sample Value
Name	Test User
Email	test@gmail.com
Phone	9999999999
Message	Testing security audit

After entering the data, the form was submitted.

Purpose

To create HTTP requests that could be captured and examined in Burp Suite.

Step 9: Capturing Requests

The submitted form generated multiple requests that appeared in Burp Suite's HTTP History section. The relevant POST request associated with the form submission was selected for analysis.

Observation

A POST request was generated when the form data was submitted to the server.

Step 10: Request Analysis

The captured request was examined in detail.

Observations

- Communication was performed using HTTPS.
- User information was transmitted through a POST request.
- Request data was structured in JSON format.
- Standard request headers such as Host, User-Agent, Content-Type, Origin, and Referer were present.

Significance

Using POST requests helps prevent sensitive information from appearing in browser URLs and history.

Step 11: Response Header Analysis

The server response headers were inspected after form submission.

Security Headers Observed

- Strict-Transport-Security (HSTS)
- Cache-Control
- Content-Type

Importance

Strict-Transport-Security (HSTS):

Ensures browsers communicate only through HTTPS, reducing the risk of SSL stripping attacks.

Cache-Control:

Controls how browser caching is handled.

Content-Type:

Specifies the format of data returned by the server.

Step 12: Backend Response Observation

During testing, the server returned an:

HTTP/2 504 Gateway Timeout

Interpretation

This response generally indicates that the server or backend service was temporarily unable to process the request within the expected time.

Possible Reasons

- Backend service delay
- Temporary server issue
- Deployment or hosting configuration problem

This observation does not directly indicate a vulnerability but may suggest availability or performance concerns.

Security Findings

Security Feature	Status
HTTPS Enabled	Yes
POST Method Used	Yes
Security Headers Present	Yes
HSTS Enabled	Yes
JSON Communication	Yes
CAPTCHA Protection	Not Observed
Multi-Factor Authentication	Not Observed
Backend Stability	504 Timeout Detected

Findings and Analysis

The website demonstrated several positive security practices. HTTPS was implemented for encrypted communication, and user data was transmitted through POST requests instead of URL parameters. Security headers such as HSTS were also present, helping improve communication security.

The application used JSON-based communication, indicating a modern web architecture. However, CAPTCHA and Multi-Factor Authentication mechanisms were not observed during testing. The absence of these controls may increase exposure to automated spam submissions or brute-force attempts.

Additionally, a 504 Gateway Timeout response was encountered during testing, suggesting temporary backend availability issues. Overall, the website showed basic security protections but could benefit from additional authentication and anti-automation measures.

Security Analysis and Vulnerability Assessment of a Visa Consultancy Website

Aim

To evaluate the security of a Visa Consultancy website by examining authentication mechanisms, identifying possible vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS), reviewing security controls, and documenting findings through manual and automated testing methods.

Objectives

- To examine the security of login and registration-related functionalities.
- To identify possible SQL Injection and XSS vulnerabilities.
- To study the implementation of input validation and sanitization techniques.
- To document detected security weaknesses and recommend solutions.
- To perform automated vulnerability scanning using security assessment tools.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Capturing and analyzing web traffic
OWASP ZAP / Nikto	Automated vulnerability scanning
Mozilla Firefox / Burp Browser	Website interaction and testing
Visa Consultancy Website	Target website for assessment
HTTPS/TLS Protocol	Secure communication analysis

Target Website

Website: <https://veerconsultancyglobal.com>

The selected website was used as a sample Visa Consultancy portal for conducting security analysis and vulnerability assessment activities.

PART 1 – SECURITY ANALYSIS OF LOGIN AND REGISTRATION SYSTEM

Step 1: Accessing the Website

The website was opened using Burp Suite's built-in browser to monitor network communication and security-related activities.

Observation

The website loaded successfully and established a secure HTTPS connection.

Step 2: Identifying Authentication Features

The website was examined to locate authentication-related components such as:

- Login pages
- Registration pages
- Inquiry forms
- Contact forms

Observation

No traditional login or registration portal was found. The website primarily contained inquiry and contact forms.

Purpose

These forms were used to study how user input is processed and transmitted to the server.

Step 3: Capturing Website Requests

Burp Suite was used to intercept and analyze requests generated when data was submitted through the website forms.

Actions Performed

1. Opened HTTP History in Burp Suite.
2. Submitted sample data through the inquiry form.
3. Examined the captured requests.

Observation

A POST request was generated when the form data was submitted.

Example:

POST /contact HTTP/2

Step 4: Request Security Analysis

The captured traffic was reviewed to understand how data was transmitted between the browser and the server.

Security Observations

Security Feature	Status
HTTPS Enabled	Yes
POST Method Used	Yes
JSON Communication	Yes
CAPTCHA Protection	Not Observed
Multi-Factor Authentication	Not Observed
Security Headers Present	Yes

Security Headers Observed

- Strict-Transport-Security (HSTS)
- Cache-Control
- Content-Type

Significance

These headers help strengthen communication security and improve protection against certain web-based attacks.

Step 5: Authentication Security Review

The website was further analyzed for authentication-related protections.

Observations

- CAPTCHA protection was not visible.
- Multi-Factor Authentication (MFA) was not implemented.
- HTTPS encryption was enabled.
- User data was transmitted using POST requests.

Potential Risks

Without CAPTCHA and MFA, the website may be more vulnerable to:

- Brute-force attacks
- Automated spam submissions
- Credential-based attacks

PART 2 – IDENTIFICATION OF SQL INJECTION AND XSS VULNERABILITIES

SQL Injection Analysis

Description

SQL Injection occurs when attackers insert malicious SQL commands into input fields with the intention of manipulating backend databases.

Example Payload

' OR '1'='1

Step 6: Testing for SQL Injection

Various input fields were tested using common SQL Injection payloads.

Test Inputs

- ' OR '1'='1
- admin' --

Areas Tested

- Contact forms
- Search fields
- Input parameters

Observation

No SQL Injection vulnerability was successfully exploited during testing. However, the absence of visible server-side validation mechanisms may increase future risk.

Possible Impact

- Unauthorized database access
- Exposure of sensitive information
- Authentication bypass
- Data modification or deletion

Cross-Site Scripting (XSS) Analysis

Description

XSS attacks occur when malicious scripts are injected into a website and executed within a user's browser.

Example Payload

Step 7: Testing for XSS Vulnerabilities

Sample XSS payloads were entered into available input fields.

Observation

The injected scripts were not executed during testing. However, insufficient input filtering or output encoding may create potential risks in the future.

Possible Impact

- Session hijacking
- Cookie theft
- Credential theft
- Malicious website redirection

PART 3 – IMPLEMENTATION OF BASIC SECURITY MEASURES

Input Validation

Description

Input validation ensures that only properly formatted and expected data is accepted by the application.

Recommended Practices

1. Restrict unnecessary special characters.
2. Validate email addresses.
3. Limit input length.
4. Reject suspicious patterns and scripts.

Examples

- Names should contain only alphabets.
- Phone numbers should contain only digits.

Input Sanitization

Description

Input sanitization removes or neutralizes potentially harmful content before processing.

Recommended Methods

1. Remove unsafe HTML tags.
2. Escape special characters.
3. Encode output before displaying it to users.

Example

Malicious scripts should be converted into harmless text instead of being executed.

Parameterized Queries

Description

Parameterized queries separate user input from SQL commands, helping prevent SQL Injection attacks.

Example

```
SELECT * FROM users WHERE email = ?
```

Additional Security Measures

Security Measure	Purpose
HTTPS Enforcement	Secure communication
CAPTCHA	Prevent automated attacks
MFA	Additional authentication layer
Security Headers	Browser-side protection
Input Validation	Prevent malicious input
Sanitization	Prevent XSS execution
Session Management	Prevent session hijacking

PART 4 – SECURITY REPORT ON DETECTED VULNERABILITIES

Detected Security Weaknesses

Vulnerability	Observation	Risk Level
Missing CAPTCHA	Not observed during testing	Medium
Missing MFA	Not implemented	Medium
Weak Input Validation	Input filtering not clearly visible	Medium
Potential SQL Injection Risk	Validation mechanisms unclear	Medium
Potential XSS Risk	Sanitization mechanisms unclear	Medium

504 Gateway Timeout	Backend instability observed	Low
---------------------	------------------------------	-----

Recommended Solutions

Vulnerability	Recommended Fix
Missing CAPTCHA	Implement Google reCAPTCHA
Missing MFA	Add OTP-based authentication
SQL Injection Risk	Use parameterized queries
XSS Risk	Apply sanitization and output encoding
Weak Input Validation	Validate all user input
Session Risks	Use secure cookies and HTTPS

PART 5 – AUTOMATED VULNERABILITY SCANNING

Step 8: Performing Automated Security Scanning

An automated vulnerability assessment was performed using the Nikto Web Vulnerability Scanner.

Tool Used

Nikto Web Vulnerability Scanner

Command

nikto -h <https://veerconsultancyglobal.com>

Step 9: Scan Results

The scanner reviewed the website for:

- Missing security headers

- Information disclosure issues
- SSL/TLS weaknesses
- Configuration vulnerabilities

Findings

1. HTTPS communication was enabled.
2. HSTS security header was present.
3. Backend timeout responses were detected.
4. Certain security configurations could be strengthened.

Importance of Automated Scanning

Automated scanning tools help identify:

- Common web vulnerabilities
- Server configuration issues
- Missing security controls
- Outdated software components

These tools improve the efficiency and effectiveness of security assessments.

Findings and Analysis

The website demonstrated several positive security practices, including HTTPS encryption, secure POST-based communication, JSON request handling, and the implementation of HSTS security headers.

During testing, SQL Injection and XSS payloads did not result in successful exploitation. However, the absence of visible input validation and sanitization controls may increase security risks if not properly implemented on the server side.

Additional improvements such as CAPTCHA protection, Multi-Factor Authentication, stronger validation mechanisms, secure session management, and continuous security monitoring would further strengthen the website's security posture.

Overall, the assessment highlighted the importance of layered security controls and regular vulnerability testing in protecting web applications from evolving cyber threats.

Authentication Security, HTTPS Implementation, Penetration Testing, and API Security Analysis of a Visa Consultancy Website

Aim

To analyze the security of a Visa Consultancy website by reviewing authentication and authorization mechanisms, studying secure password management practices, understanding HTTPS implementation, performing penetration testing on authentication and session handling, and researching API security best practices.

Objectives

- To evaluate authentication and authorization mechanisms for potential security weaknesses.
- To understand secure password storage using modern hashing algorithms.
- To study HTTPS implementation and secure communication practices.
- To perform penetration testing related to authentication and session management.
- To research and document best practices for securing APIs.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Request interception and penetration testing
Mozilla Firefox / Burp Browser	Website interaction and analysis
OpenSSL / XAMPP / Localhost	HTTPS configuration
bcrypt / Argon2	Secure password hashing
Visa Consultancy Website	Target website for security assessment

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected as a representative Visa Processing and Consultancy platform suitable for studying authentication security, HTTPS communication, penetration testing, and API security concepts.

PART 1 – REVIEW OF AUTHENTICATION AND AUTHORIZATION MECHANISMS

Introduction

Authentication is the process of verifying a user's identity, while authorization determines what resources or actions a user is allowed to access after successful authentication.

Weak authentication and authorization controls can lead to:

- Unauthorized access
- Account compromise
- Data leakage
- Privilege escalation attacks

Step 1: Authentication Analysis

The website was examined to identify authentication-related security controls and protections.

Security Features Observed

Security Feature	Status
HTTPS Enabled	Yes
Login System Available	Limited / Not Clearly Visible
Multi-Factor Authentication (MFA)	Not Observed
CAPTCHA Protection	Not Observed

Session Cookies	Present
Security Headers	Present

Observation

The website used HTTPS for secure communication and maintained session cookies. However, no visible CAPTCHA or Multi-Factor Authentication mechanism was observed during testing.

Step 2: Authorization Analysis

The website was reviewed to understand how access control and authorization mechanisms were implemented.

Observations

1. No visible role-based access system was detected.
2. Public pages were accessible without authentication.
3. Session behavior was monitored using Burp Suite.

Potential Risks

- Unauthorized access
- Session hijacking
- Broken access control vulnerabilities

Authentication Weaknesses Identified

Weakness	Security Risk
No MFA	Increased risk of credential attacks
No CAPTCHA	Automated spam and brute-force attacks
Limited Authentication Visibility	Potential weak access control
Session Handling Exposure	Possibility of session hijacking

Recommended Improvements

1. Implement Multi-Factor Authentication (MFA).
2. Add CAPTCHA protection to sensitive forms.
3. Enforce secure session expiration policies.
4. Implement Role-Based Access Control (RBAC).
5. Use Secure and HttpOnly cookie attributes.

PART 2 – IMPLEMENTATION OF SECURE PASSWORD POLICIES

Introduction

Passwords are one of the primary security mechanisms used to protect user accounts. Weak password practices can expose users to unauthorized access and data breaches.

Modern password security relies on strong password policies and secure hashing algorithms such as bcrypt and Argon2.

Recommended Password Policy

The following password requirements are recommended:

1. Minimum length of 12 characters.
2. Combination of:
 - Uppercase letters
 - Lowercase letters
 - Numbers
 - Special characters
3. Password expiration policies.
4. Prevention of password reuse.
5. Password strength validation.

Examples of Strong Passwords

Example 1:

Visa@Secure2026

Example 2:

Str0ng#PortalAccess

Password Hashing Using bcrypt

Description

bcrypt is a secure password hashing algorithm designed specifically for protecting user credentials.

Advantages

- Slows down brute-force attacks.
- Automatically applies salting.
- Widely adopted in modern web applications.

Example bcrypt Hash

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

Password Hashing Using Argon2

Description

Argon2 is a modern password hashing algorithm that offers strong protection against password cracking attempts.

Advantages

- Resistant to GPU-based attacks.
- Memory-hard design improves security.
- Provides stronger protection against brute-force attacks.

Importance of Password Hashing

Password hashing protects user credentials even if:

- A database breach occurs.
- Attackers gain unauthorized access to stored data.

Plain-text passwords should never be stored in any database.

Recommended Password Security Measures

Security Measure	Purpose
bcrypt / Argon2 Hashing	Secure password storage
MFA	Additional authentication layer
CAPTCHA	Prevent automated attacks
Account Lockout	Reduce brute-force attacks
Password Strength Meter	Encourage stronger passwords

PART 3 – HTTPS CONFIGURATION AND ITS IMPORTANCE

Introduction

HTTPS (HyperText Transfer Protocol Secure) uses SSL/TLS encryption to secure communication between users and web servers.

During testing, HTTPS communication was observed on the Visa Consultancy website.

Importance of HTTPS

HTTPS helps protect against:

- Man-in-the-Middle (MITM) attacks
- Credential theft
- Data interception
- Session hijacking

Step 3: HTTPS Setup in a Local Environment

Method Used

A local HTTPS environment was conceptually configured using development tools such as XAMPP and SSL certificates.

Steps Performed

1. Installed XAMPP and Apache Server.
2. Enabled SSL support.
3. Generated a self-signed SSL certificate.
4. Configured an HTTPS virtual host.
5. Restarted the Apache service.
6. Accessed the website using:

<https://localhost>

Example HTTPS Configuration

```
<VirtualHost *:443>  
    DocumentRoot "C:/xampp/htdocs"  
    ServerName localhost  
    SSLEngine on  
    SSLCertificateFile "server.crt"  
    SSLCertificateKeyFile "server.key"  
</VirtualHost>
```

Security Benefits of HTTPS

Benefit	Description
Encryption	Protects transmitted data
Authentication	Verifies server identity
Data Integrity	Prevents unauthorized modification
Secure Sessions	Protects cookies and session data

Practical Observation

During Burp Suite testing:

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. User data was transmitted securely using POST requests.

PART 4 – PENETRATION TESTING OF AUTHENTICATION AND SESSION HANDLING

Introduction

Penetration testing helps identify weaknesses in authentication systems and session management mechanisms.

Burp Suite was used to inspect requests, responses, cookies, and session-related behavior.

Step 4: Capturing Requests

Actions Performed

1. Opened HTTP History in Burp Suite.
2. Submitted form requests.
3. Reviewed session cookies and request handling.

Observed Request

POST /contact HTTP/2

Step 5: Session Security Analysis

The following session-related controls were examined:

Session Security Check	Observation
HTTPS Enabled	Yes
Secure Cookies	Partially Observed
Session Tokens	Present
Session Timeout	Not Clearly Visible
HSTS Enabled	Yes

Step 6: Authentication Penetration Testing

The following tests were conceptually performed:

1. Session cookie inspection.
2. Repeated request analysis.
3. Authentication request review.
4. Brute-force protection assessment.

Findings

1. HTTPS reduced the risk of traffic interception.
2. CAPTCHA protection was not visible.
3. MFA was not observed.
4. Session protection mechanisms were only partially visible.

Potential Risks

- Session hijacking
- Credential attacks
- Automated brute-force attempts
- Unauthorized access

Recommended Security Improvements

1. Use secure session cookies.
2. Enable HttpOnly and Secure flags.
3. Regenerate session IDs after authentication.
4. Implement MFA and CAPTCHA.
5. Configure automatic session expiration.

PART 5 – BEST PRACTICES FOR SECURING APIs

Introduction

Modern web applications rely heavily on APIs to exchange information between frontend and backend systems.

During testing, JSON-based API communication was observed.

Improper API security can expose:

- User information
- Authentication tokens
- Backend services

Common API Security Risks

Risk	Description
Broken Authentication	Weak token handling
Broken Authorization	Unauthorized access
Injection Attacks	Malicious input execution
Excessive Data Exposure	Leakage of sensitive information
Insecure Endpoints	Unprotected API access

Recommended API Security Practices

1. Use HTTPS for APIs

All API communication should be encrypted using HTTPS.

2. Implement Secure Authentication

Use secure authentication methods such as:

- JWT Tokens
- OAuth 2.0
- API Keys

3. Validate API Inputs

Validate and sanitize all incoming parameters to prevent:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection

4. Apply Rate Limiting

Rate limiting helps prevent:

- API abuse
- DDoS attacks
- Automated requests

5. Restrict Access

Implement:

- Role-Based Access Control (RBAC)
- Permission validation
- Access tokens

6. Secure Error Handling

Avoid exposing:

- Server details
- Database information
- Internal configurations

7. Logging and Monitoring

Monitor:

- Failed requests
- Suspicious traffic
- Unauthorized access attempts

API Security Recommendations

Security Practice	Purpose
HTTPS APIs	Secure communication
JWT/OAuth Authentication	Secure access control
Input Validation	Prevent injection attacks
Rate Limiting	Prevent abuse
RBAC	Restrict unauthorized access
Secure Logging	Detect attacks
Error Handling	Prevent information leakage

Findings and Analysis

The website demonstrated the use of HTTPS communication, secure POST-based request handling, and security headers such as HSTS. These controls improve communication security and help protect user data during transmission.

However, several security improvements are recommended, including:

- Multi-Factor Authentication (MFA)
- CAPTCHA protection
- Stronger session security controls
- Enhanced access control mechanisms
- Additional API security measures

Secure password storage using bcrypt or Argon2 is strongly recommended to protect user credentials in the event of a database compromise.

The activity emphasized the importance of layered security controls for authentication systems, session management, and API communication.

Conclusion

This practical activity provided hands-on exposure to authentication analysis, secure password management, HTTPS implementation concepts, penetration testing techniques, and API security practices.

The exercise improved understanding of:

- Authentication and authorization security
- Password hashing mechanisms
- HTTPS communication
- Session management protections
- Penetration testing methodologies
- API security best practices

Strong cybersecurity practices are essential for protecting sensitive user information and maintaining secure online services in modern web applications.

Secure Coding Techniques and Security Code Review of a Visa Consultancy Website

Aim

To study and implement secure coding practices by analyzing user input handling, password encryption methods, secure error management, and code review techniques for a Visa Consultancy website. The activity also focuses on understanding industry best practices for developing secure web applications.

Objectives

- To implement input validation techniques for preventing SQL Injection and XSS attacks.
- To study secure password encryption using bcrypt and Argon2.
- To understand secure error handling mechanisms.
- To perform a security review of a sample login module.
- To research best practices for maintaining secure application code.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Security testing and request analysis
Mozilla Firefox / Burp Browser	Website interaction and testing
bcrypt / Argon2	Password encryption and hashing
Sample Login Module	Security code review
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform that is suitable for studying secure coding practices and web application security concepts.

PART 1 – IMPLEMENTATION OF INPUT VALIDATION

Introduction

Input validation is one of the most important security controls in web application development. Improper handling of user input can expose applications to several security threats, including:

- SQL Injection
- Cross-Site Scripting (XSS)
- Command Injection
- Data Manipulation Attacks

The user input fields available on the Visa Consultancy website were analyzed to understand how user data is handled and protected.

Step 1: Identifying User Input Fields

The following fields were identified during testing:

- Contact forms
- Inquiry forms
- Email fields
- Phone number fields
- Message fields

Purpose

These input fields were reviewed to determine whether appropriate validation and sanitization controls were implemented.

Step 2: SQL Injection Testing

Common SQL Injection payloads were conceptually tested against available input fields.

Test Payloads

- ' OR '1'='1
- admin' --

Purpose

To determine whether malicious SQL commands could influence backend database operations.

Observation

No direct SQL Injection vulnerability was identified during testing. However, implementing strong server-side validation remains essential to reduce future security risks.

Step 3: Cross-Site Scripting (XSS) Testing

Sample XSS payloads were entered into available form fields.

Test Payloads

```
<script>alert('XSS')</script>
```

```
<img src=x onerror=alert(1)>
```

Purpose

To verify whether malicious scripts could execute within the user's browser.

Observation

The injected scripts were not executed during testing. Nevertheless, insufficient sanitization controls may create future XSS risks if not properly managed.

Recommended Input Validation Techniques

Validation Technique	Purpose
Input Length Restriction	Prevent oversized payloads
Email Format Validation	Accept valid email addresses only
Numeric Validation	Restrict phone fields to numbers
Special Character Filtering	Prevent injection attacks
Server-Side Validation	Prevent bypassing client-side checks

Example Validation Rules

1. Name fields should allow only alphabetic characters.
2. Phone number fields should accept only numeric values.
3. Email addresses should follow standard email formats.
4. Scripts and suspicious characters should be filtered before processing.

Input Sanitization

Input sanitization removes or neutralizes potentially dangerous content before processing user data.

Example

Malicious code such as:

`<script>`

should be converted into harmless text instead of being executed by the browser.

Importance of Input Validation

Effective validation and sanitization help protect against:

- SQL Injection attacks
- XSS vulnerabilities
- Data corruption
- Unauthorized command execution

PART 2 – PASSWORD ENCRYPTION USING bcrypt AND Argon2

Introduction

Passwords should never be stored in plain text because a database compromise could expose sensitive user credentials.

Secure hashing algorithms such as bcrypt and Argon2 provide strong protection by converting passwords into irreversible hashes before storage.

bcrypt Password Hashing

Description

bcrypt is a widely used password hashing algorithm specifically designed for secure credential storage.

Advantages

1. Automatically applies salting.
2. Resistant to brute-force attacks.
3. Uses a computationally intensive hashing process.

Example bcrypt Hash

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

Argon2 Password Hashing

Description

Argon2 is a modern password hashing algorithm recommended for advanced password protection.

Advantages

1. Memory-hard design improves security.
2. Resistant to GPU-based cracking attacks.
3. Provides strong protection against brute-force attempts.

Importance of Password Encryption

Password hashing protects user credentials even if:

- Database leaks occur.
- Attackers gain unauthorized access to stored data.

For this reason, plain-text password storage should never be used in any application.

Recommended Password Security Practices

Security Practice	Purpose
bcrypt / Argon2 Hashing	Secure password storage
Strong Password Policy	Reduce weak password usage
Multi-Factor Authentication	Additional security layer
Password Strength Meter	Encourage stronger passwords
Password Expiration	Reduce long-term credential exposure

Examples of Strong Passwords

Example 1:

Visa@Secure2026

Example 2:

Str0ng#PortalAccess

PART 3 – IMPLEMENTATION OF SECURE ERROR HANDLING

Introduction

Improper error handling can expose sensitive information such as:

- Database details
- Server paths
- Internal configurations
- Application logic

Attackers often use this information to identify weaknesses and launch targeted attacks.

Secure Error Handling Principles

1. Avoid displaying technical system errors to users.
2. Use generic and user-friendly error messages.
3. Log detailed error information securely on the server.
4. Hide database and framework details from end users.

Example of an Insecure Error Message

SQL syntax error near line 1 in database users

Security Risk

This message exposes backend database information that could assist attackers.

Example of a Secure Error Message

An unexpected error occurred. Please try again later.

Recommended Error Handling Practices

Practice	Purpose
Generic Error Messages	Prevent information leakage
Secure Logging	Store detailed logs safely
Exception Handling	Manage unexpected errors securely
Hide Stack Traces	Protect internal application details

Importance of Secure Error Handling

Proper error management helps prevent:

- Information disclosure
- Reconnaissance attacks
- Exploitation of backend systems

PART 4 – SECURITY CODE REVIEW OF A SAMPLE LOGIN MODULE

Introduction

Security code reviews help identify insecure coding practices before an application is deployed.

A sample login module was conceptually reviewed to identify common security weaknesses.

Weaknesses Identified

Weakness	Security Risk
Plain-text Password Storage	Credential exposure
Missing Input Validation	Injection attacks
Weak Session Handling	Session hijacking
Missing Password Hashing	Unauthorized access
Detailed Error Messages	Information leakage
No MFA	Credential attack risk

Recommended Security Improvements

- 1. Implement bcrypt or Argon2 password hashing.
- 2. Validate and sanitize all user inputs.
- 3. Use secure session management techniques.
- 4. Display generic error messages.
- 5. Enable Multi-Factor Authentication.
- 6. Use HTTPS for all login-related communication.

Secure Login Module Recommendations

Security Feature	Purpose
HTTPS Login	Secure communication
Password Hashing	Protect credentials
Input Validation	Prevent injection attacks
Secure Cookies	Protect session data
MFA	Additional account protection
Session Timeout	Reduce session hijacking risk

PART 5 – FIVE BEST PRACTICES FOR MAINTAINING SECURE CODE

1. Validate and Sanitize All User Inputs

Every user input should be validated and sanitized before processing. This helps prevent:

- SQL Injection attacks
- Cross-Site Scripting (XSS)
- Execution of malicious payloads

2. Use Secure Password Hashing

Passwords should always be protected using secure hashing algorithms such as:

- bcrypt
- Argon2

Plain-text password storage should never be permitted.

3. Implement Strong Authentication and Authorization

Applications should use:

- Multi-Factor Authentication (MFA)
- Role-Based Access Control (RBAC)
- Secure session management

These controls reduce the risk of unauthorized access.

4. Use HTTPS for Secure Communication

All web communication should be encrypted using HTTPS to protect:

- User credentials
- Session cookies
- Sensitive personal information

5. Apply Secure Error Handling and Logging

Applications should:

- Hide technical system information from users.
- Maintain secure server-side logs.
- Prevent exposure of internal configurations and application details.

Practical Observations During Testing

During the analysis of the Visa Consultancy website, the following observations were made:

1. HTTPS communication was enabled.
2. POST requests were used to transmit form data.
3. HSTS security headers were present.
4. CAPTCHA protection was not visible.
5. Multi-Factor Authentication was not observed.
6. Input filtering mechanisms were not clearly visible.

Findings and Analysis

The practical activity highlighted the importance of secure coding practices in protecting web applications against modern cyber threats.

Input validation and sanitization play a critical role in reducing SQL Injection and XSS risks, while secure password hashing algorithms such as bcrypt and Argon2 strengthen credential security.

Secure error handling minimizes information disclosure, and regular code reviews help identify vulnerabilities before deployment.

Implementing these practices significantly improves the overall security posture of Visa Processing and Consultancy web applications.

Conclusion

This practical successfully demonstrated secure coding techniques and security code review concepts for a Visa Consultancy website.

The activity improved understanding of:

- Input validation techniques
- SQL Injection prevention
- XSS prevention
- Password encryption and hashing
- Secure error handling
- Secure coding principles
- Security code review methodologies

Adopting secure coding standards is essential for protecting sensitive user information, reducing cyber risks, and maintaining reliable and secure web applications.

Network Security Analysis and Protection of a Visa Consultancy Website

Aim

To study and enhance the network security of a Visa Consultancy website by analyzing web server security practices, configuring firewall protection, examining HTTPS implementation, identifying common network vulnerabilities, and documenting security recommendations for improving overall protection.

Objectives

- To research best practices for securing web servers.
- To configure and test firewall protection on a local system.
- To analyze HTTPS implementation and understand its security benefits.
- To identify common network vulnerabilities and recommend solutions.
- To document network security recommendations for improving website security.

Tools Used

Tool	Purpose
Burp Suite Community Edition	HTTPS and traffic analysis
Windows Firewall / UFW	Firewall configuration
Mozilla Firefox / Burp Browser	Website interaction
Visa Consultancy Website	Security analysis target
HTTPS/TLS Protocol	Secure communication analysis

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for studying network security concepts and protection mechanisms.

PART 1 – FIVE BEST PRACTICES FOR SECURING A WEB SERVER

Introduction

Web servers host websites and process user requests over the internet. If a web server is not properly secured, attackers may exploit vulnerabilities to gain unauthorized access, steal sensitive information, or disrupt services.

Securing a web server is essential for protecting:

- User information
- Authentication systems
- Backend applications
- APIs
- Network infrastructure

Best Practice 1 – Use HTTPS and SSL/TLS Encryption

Description

HTTPS uses SSL/TLS encryption to secure communication between users and web servers.

Benefits

1. Prevents data interception.
2. Protects login credentials.
3. Reduces the risk of Man-in-the-Middle (MITM) attacks.
4. Secures session cookies and sensitive information.

Recommended Actions

1. Use TLS 1.2 or TLS 1.3.
2. Redirect all HTTP traffic to HTTPS.
3. Enable HSTS security headers.
4. Use valid SSL/TLS certificates.

Best Practice 2 – Keep Server Software Updated

Description

Outdated software often contains known vulnerabilities that attackers can exploit.

Recommended Actions

1. Apply security patches regularly.
2. Update web server software such as Apache or Nginx.
3. Remove unused applications and services.
4. Monitor newly discovered vulnerabilities.

Benefits

- Reduces exposure to known security flaws.
- Improves server performance and stability.

Best Practice 3 – Configure Firewalls Properly

Description

Firewalls control incoming and outgoing network traffic based on predefined security rules.

Recommended Actions

1. Allow only necessary ports and services.
2. Block unused or risky ports.
3. Restrict unauthorized remote access.

4. Monitor suspicious network activity.

Benefits

- Prevents unauthorized access.
- Reduces the attack surface of the server.

Best Practice 4 – Use Strong Authentication and Access Control

Description

Weak authentication mechanisms can increase the risk of unauthorized access.

Recommended Actions

1. Use strong passwords.
2. Implement Multi-Factor Authentication (MFA).
3. Apply Role-Based Access Control (RBAC).
4. Disable default administrator accounts.

Benefits

- Protects administrative systems.
- Reduces credential-based attacks.

Best Practice 5 – Enable Logging and Monitoring

Description

Continuous monitoring helps identify suspicious activities and potential security incidents.

Recommended Actions

1. Maintain access and activity logs.
2. Monitor failed login attempts.
3. Detect unusual traffic patterns.

4. Configure automated security alerts.

Benefits

- Enables early detection of attacks.
- Improves incident response capabilities.

Web Server Security Best Practices Summary

Best Practice	Purpose
HTTPS/TLS Encryption	Secure communication
Regular Updates	Prevent known vulnerabilities
Firewall Configuration	Restrict unauthorized access
Strong Authentication	Protect administrative systems
Logging and Monitoring	Detect attacks and suspicious activities

PART 2 – FIREWALL CONFIGURATION AND TESTING

Introduction

A firewall acts as a protective barrier between a system and the network by filtering traffic based on security rules.

Step 1: Accessing Firewall Settings

Windows Firewall was used to configure and test firewall rules.

Actions Performed

1. Opened Windows Defender Firewall.
2. Accessed Advanced Settings.
3. Navigated to firewall rule configuration.

Purpose

To create security rules that control network access.

Step 2: Creating a Basic Firewall Rule

A basic inbound firewall rule was created to block traffic on a specific port.

Configuration Steps

1. Selected Inbound Rules.
2. Created a New Rule.
3. Chose Port-based filtering.
4. Selected TCP protocol.
5. Specified port **8080**.
6. Selected **Block the Connection**.
7. Applied the rule to:
 - Domain
 - Private
 - Public networks
8. Saved the rule.

Purpose

To prevent unauthorized communication through port 8080.

Step 3: Testing the Firewall Rule

The configured rule was tested by attempting communication through the blocked port.

Observation

Traffic on port 8080 was successfully restricted after applying the firewall rule.

Importance of Firewall Rules

Firewall rules help:

- Restrict unauthorized access
- Prevent network attacks
- Protect local services
- Minimize the attack surface

PART 3 – HTTPS ANALYSIS AND ITS IMPORTANCE

Introduction

HTTPS (HyperText Transfer Protocol Secure) protects communication between users and websites using SSL/TLS encryption.

Step 4: Verifying HTTPS Usage

The Visa Consultancy website was opened using Burp Browser.

Website Tested

<https://veerconsultancyglobal.com>

Observation

The website successfully established a secure HTTPS connection.

Security Headers Observed

The following security header was identified:

- Strict-Transport-Security (HSTS)

Importance of HTTPS

HTTPS protects against:

- Credential theft
- Data interception
- Session hijacking
- Man-in-the-Middle attacks

Benefits of HTTPS

Benefit	Description
Encryption	Protects transmitted information
Authentication	Verifies server identity
Data Integrity	Prevents unauthorized modification
Secure Sessions	Protects cookies and session data

Practical Observation

During Burp Suite testing:

1. HTTPS communication was enabled.
2. POST requests transmitted data securely.
3. HSTS security headers were present.

PART 4 – COMMON NETWORK VULNERABILITIES AND SOLUTIONS

Introduction

Network vulnerabilities are weaknesses that attackers may exploit to compromise systems, networks, or sensitive information.

Vulnerability 1 – Open Ports

Description

Unnecessary open ports may expose services to attackers.

Risks

- Unauthorized access
- Remote exploitation

Recommended Solution

1. Close unused ports.
2. Restrict access through firewall rules.

Vulnerability 2 – Weak Passwords

Description

Weak passwords can be easily guessed or cracked through brute-force attacks.

Risks

- Unauthorized account access
- Account compromise

Recommended Solution

1. Implement strong password policies.
2. Enable Multi-Factor Authentication (MFA).

Vulnerability 3 – Unencrypted Communication

Description

HTTP transmits information without encryption.

Risks

- Credential theft
- Data interception

Recommended Solution

1. Use HTTPS/TLS encryption.
2. Enable HSTS security headers.

Vulnerability 4 – Improper Firewall Configuration

Description

Weak firewall rules may expose systems to external threats.

Risks

- Unauthorized access
- Malware communication

Recommended Solution

1. Restrict unnecessary ports.
2. Regularly review firewall logs.

Vulnerability 5 – Outdated Software

Description

Older software versions may contain publicly known security vulnerabilities.

Risks

- Exploitation of software flaws
- Server compromise

Recommended Solution

1. Apply security updates regularly.
2. Upgrade server software when necessary.

Network Vulnerabilities Summary

Vulnerability	Risk	Solution
Open Ports	Unauthorized access	Close unused ports
Weak Passwords	Credential attacks	Strong passwords and MFA
Unencrypted Communication	Data interception	HTTPS/TLS encryption
Weak Firewall Rules	Network exposure	Secure firewall configuration
Outdated Software	Exploitation risk	Regular software updates

PART 5 – NETWORK SECURITY RECOMMENDATIONS

Recommended Security Improvements

1. Implement HTTPS Everywhere

All webpages and APIs should use HTTPS encryption to protect user data during transmission.

2. Configure Secure Firewall Rules

Allow only required network ports and block unnecessary traffic.

3. Enable Multi-Factor Authentication (MFA)

Adding an additional authentication layer improves account security.

4. Apply Regular Security Updates

Servers and applications should be updated regularly to eliminate known vulnerabilities.

5. Enable Security Monitoring

Monitor:

- Failed login attempts
- Suspicious traffic
- Unauthorized access attempts

6. Use Strong Password Policies

Passwords should:

- Be at least 12 characters long
- Include uppercase and lowercase letters
- Contain numbers and special characters

7. Implement Secure Session Management

Use:

- Secure cookies
- HttpOnly flags
- Automatic session expiration

8. Restrict Administrative Access

Administrative functions should only be accessible to authorized users.

Network Security Recommendations Summary

Recommendation	Purpose
HTTPS Encryption	Secure communication
Firewall Protection	Restrict unauthorized access
MFA	Protect user accounts
Security Updates	Reduce vulnerabilities
Monitoring and Logging	Detect attacks
Strong Passwords	Prevent credential attacks
Secure Sessions	Prevent session hijacking

Practical Observations During Testing

During the analysis of the Visa Consultancy website using Burp Suite, the following observations were made:

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. POST requests were used to transmit user data securely.
4. Multi-Factor Authentication (MFA) was not visible.
5. CAPTCHA protection was not observed.
6. A 504 Gateway Timeout response was encountered, indicating temporary backend or deployment-related issues.

Findings and Analysis

The website demonstrated secure HTTPS communication and implemented important security headers such as HSTS, which improve communication security and protect transmitted data.

However, additional improvements such as stronger authentication controls, firewall hardening, security monitoring, and secure session management would further strengthen the website's network security posture.

Proper firewall configuration, HTTPS implementation, regular software updates, and continuous monitoring are essential for defending web servers and online portals against network-based cyber threats.

Conclusion

This practical successfully demonstrated network security analysis and protection techniques for a Visa Consultancy website.

The activity improved understanding of:

- Web server security practices
- Firewall configuration and testing
- HTTPS communication
- Common network vulnerabilities
- Secure networking techniques
- Security monitoring and protection

Implementing strong network security controls is essential for protecting sensitive user information, maintaining service availability, and ensuring the security of modern web applications.

Web Server Security Analysis and Hardening of a Visa Consultancy Website

Aim

To study and improve the security of the web server hosting a Visa Consultancy website by researching secure server practices, examining software update procedures, analyzing security headers, understanding logging and monitoring mechanisms, and documenting security findings and recommendations.

Objectives

- To research important web server security best practices.
- To understand the importance of regular server software updates.
- To analyze security headers implemented by the website.
- To study logging and monitoring techniques used for threat detection.
- To document findings and recommend security improvements.

Tools Used

Tool	Purpose
Burp Suite Community Edition	HTTP/HTTPS traffic analysis
Browser Developer Tools (F12)	Security header inspection
Mozilla Firefox / Burp Browser	Website interaction
Apache / Nginx Research	Web server security study
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for studying web server security concepts and hardening techniques.

PART 1 – SECURITY BEST PRACTICES FOR WEB SERVERS

Introduction

Web servers play a critical role in hosting websites and handling communication between users and applications. If not properly secured, they may become vulnerable to cyberattacks, resulting in unauthorized access, data breaches, malware infections, or service disruptions.

Strong security practices are essential for protecting:

- User information
- Authentication systems
- Backend applications
- APIs
- Network resources

Best Practice 1 – Use HTTPS and TLS Encryption

Description

HTTPS uses SSL/TLS encryption to secure communication between users and web servers.

Benefits

1. Protects sensitive information during transmission.
2. Prevents credential theft.

3. Reduces the risk of Man-in-the-Middle (MITM) attacks.
4. Secures session cookies and authentication data.

Recommended Actions

1. Use TLS 1.2 or TLS 1.3.
2. Redirect all HTTP traffic to HTTPS.
3. Enable HSTS security headers.
4. Use valid SSL/TLS certificates.

Best Practice 2 – Keep Server Software Updated

Description

Outdated software often contains known vulnerabilities that attackers can exploit.

Recommended Actions

1. Regularly update Apache or Nginx.
2. Apply operating system security patches.
3. Remove unnecessary applications and services.
4. Monitor vulnerability reports and security advisories.

Benefits

- Reduces exposure to known attacks.
- Improves performance and reliability.
- Strengthens overall server security.

Best Practice 3 – Configure Strong Access Control

Description

Improper access control can allow unauthorized users to access sensitive server resources.

Recommended Actions

1. Use strong administrator passwords.
2. Enable Multi-Factor Authentication (MFA).
3. Restrict administrative access.
4. Implement Role-Based Access Control (RBAC).

Benefits

- Prevents unauthorized access.
- Enhances account security.
- Reduces insider and external threats.

Best Practice 4 – Enable Logging and Monitoring

Description

Continuous monitoring helps detect suspicious activities and security incidents before they become serious threats.

Recommended Actions

1. Enable server access logs.
2. Monitor failed login attempts.
3. Detect unusual traffic patterns.
4. Configure automated alerts.

Benefits

- Early detection of attacks.
- Faster incident response.
- Improved security visibility.

Best Practice 5 – Configure Security Headers

Description

Security headers provide additional protection against browser-based attacks.

Important Security Headers

Header	Purpose
Content-Security-Policy (CSP)	Prevents XSS attacks
X-Frame-Options	Prevents clickjacking
Strict-Transport-Security (HSTS)	Forces HTTPS usage
X-Content-Type-Options	Prevents MIME-sniffing attacks

Benefits

- Improves browser security.
- Reduces client-side attack risks.
- Strengthens website protection.

Web Server Security Best Practices Summary

Best Practice	Purpose
HTTPS/TLS Encryption	Secure communication
Regular Updates	Prevent known vulnerabilities
Strong Access Control	Protect sensitive resources

Logging and Monitoring	Detect suspicious activities
Security Headers	Improve browser security

PART 2 – CHECKING SERVER SOFTWARE AND UPDATES

Introduction

Web servers such as Apache and Nginx require regular updates to maintain security and performance.

Step 1: Identifying Server Technology

The website was analyzed using browser developer tools and HTTP response headers.

Observation

The exact server software version was not publicly exposed. This is considered a good security practice because it limits information disclosure to potential attackers.

Possible Server Technologies

- Apache
- Nginx
- Cloud-hosted server platforms

Importance of Server Updates

Outdated server software may contain:

- Known vulnerabilities
- Remote code execution flaws
- Authentication bypass weaknesses

Safe Update Procedures

Apache Update Example (Linux)

```
sudo apt update  
sudo apt upgrade apache2
```

Nginx Update Example (Linux)

```
sudo apt update  
sudo apt upgrade nginx
```

Recommended Update Practices

- 1. Back up server configurations before updating.
- 2. Test updates in a staging environment.
- 3. Schedule updates during maintenance periods.
- 4. Restart services safely after applying updates.

Benefits of Regular Updates

Benefit	Description
Patch Vulnerabilities	Fix known security flaws
Improve Stability	Reduce failures and downtime
Improve Performance	Increase server efficiency
Reduce Exploitation Risk	Protect against attacks

PART 3 – SECURITY HEADER ANALYSIS

Introduction

Security headers help browsers enforce additional protections against common web attacks such as:

- Cross-Site Scripting (XSS)
- Clickjacking
- MIME-sniffing attacks

Step 2: Inspecting Security Headers

Browser developer tools were used to inspect HTTP response headers.

Steps Performed

1. Opened the website.
2. Pressed **F12** to open Developer Tools.
3. Navigated to **Network** → **Headers**.
4. Reviewed the response headers.

Security Headers Observed

Security Header	Observation
Strict-Transport-Security	Present
Content-Type	Present
Cache-Control	Present
Content-Security-Policy	Not Clearly Visible
X-Frame-Options	Not Clearly Visible

Importance of Key Security Headers

Content-Security-Policy (CSP)

Helps prevent execution of unauthorized scripts and reduces XSS risks.

X-Frame-Options

Protects against clickjacking by preventing websites from being embedded within frames.

Strict-Transport-Security (HSTS)

Forces browsers to use secure HTTPS connections.

Example Security Header Configuration

Apache Configuration

Header always set X-Frame-Options "DENY"

Header always set Content-Security-Policy "default-src 'self';"

Nginx Configuration

```
add_header X-Frame-Options "DENY";
```

```
add_header Content-Security-Policy "default-src 'self';";
```

Benefits of Security Headers

1. Reduce XSS risks.
2. Prevent clickjacking attacks.
3. Improve browser security.
4. Minimize client-side vulnerabilities.

PART 4 – LOGGING AND MONITORING

Introduction

Logging and monitoring are essential for identifying suspicious activities, detecting attacks, and investigating security incidents.

Step 3: Studying Logging Mechanisms

Web servers typically maintain several types of logs, including:

- Access logs
- Error logs
- Authentication logs

Apache Log Locations

`/var/log/apache2/access.log`

`/var/log/apache2/error.log`

Nginx Log Locations

`/var/log/nginx/access.log`

`/var/log/nginx/error.log`

Importance of Logging

Logs help identify:

- Failed login attempts
- Suspicious traffic patterns
- Unauthorized access attempts
- DDoS attacks
- Brute-force attacks

Step 4: Researching Log Analysis Tools

Several tools are available for log management and security monitoring.

Tool	Purpose
Splunk	Log analysis and monitoring
ELK Stack	Centralized logging
Fail2Ban	Blocks repeated failed login attempts
Graylog	Security event monitoring

Benefits of Fail2Ban

1. Detects repeated failed login attempts.
2. Automatically blocks suspicious IP addresses.
3. Helps prevent brute-force attacks.

Monitoring Recommendations

1. Continuously monitor server activity.
2. Review authentication failures regularly.
3. Enable automated alerts.
4. Store logs securely for future analysis.

PART 5 – FINDINGS, FIXES, AND RECOMMENDATIONS

Security Findings

Issue Identified	Observation
HTTPS Enabled	Yes
HSTS Header Present	Yes
Content-Security-Policy	Not Clearly Visible
X-Frame-Options	Not Clearly Visible
Server Software Details Hidden	Yes
Logging Mechanisms	Research Completed

Recommended Fixes

Security Issue	Recommended Fix
Missing CSP Header	Add Content-Security-Policy
Missing X-Frame-Options	Enable clickjacking protection
Weak Monitoring	Implement centralized logging
Potential Brute-Force Risk	Use Fail2Ban and MFA
Server Exposure Risks	Keep software updated

Additional Security Recommendations

1. Use HTTPS across all webpages and APIs.
2. Implement Content-Security-Policy headers.
3. Enable X-Frame-Options protection.
4. Keep Apache, Nginx, and operating systems updated.
5. Monitor logs continuously.
6. Restrict administrative access.
7. Enable Multi-Factor Authentication (MFA).
8. Use firewalls and rate-limiting controls.

Practical Observations During Testing

During the analysis of the Visa Consultancy website:

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. POST requests transmitted data securely.
4. Some advanced security headers were not clearly visible.
5. Server software details were partially hidden.

These observations indicate that the website implements basic security protections, although additional hardening measures could further improve security.

Findings and Analysis

The practical demonstrated the importance of web server security in protecting sensitive Visa Processing and Consultancy systems. HTTPS communication and HSTS headers improved communication security, while logging and monitoring mechanisms provide visibility into potential threats and unauthorized activities.

Additional protections such as Content-Security-Policy headers, X-Frame-Options, centralized logging, and continuous server updates would further strengthen the overall security posture of the web server.

Conclusion

This practical successfully demonstrated web server security analysis and hardening concepts for a Visa Consultancy website.

The activity improved understanding of:

- Web server security best practices
- Server update procedures
- Security headers
- Logging and monitoring techniques
- Threat detection methods
- Secure server configuration

Implementing strong web server security measures is essential for protecting sensitive user information, maintaining service availability, and ensuring secure online operations.

Database Security Analysis and Protection of a Visa Consultancy Website

Aim

To strengthen the security of the database used by a Visa Consultancy website by analyzing user access control, securing authentication mechanisms, preventing SQL Injection attacks, implementing data encryption, studying logging and backup strategies, and documenting security findings and recommendations.

Objectives

- To analyze database user access control and permissions.
- To study secure database authentication mechanisms.
- To understand SQL Injection prevention techniques.
- To examine encryption methods for protecting sensitive data.
- To research database logging and backup strategies.
- To document security findings, fixes, and recommendations.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Request analysis and SQL Injection testing
MySQL / PostgreSQL Research	Database security study
bcrypt / Argon2	Password hashing and encryption
SQL Queries	Database access analysis
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for database security analysis and cybersecurity research.

PART 1 – ANALYZING USER ACCESS CONTROL

Introduction

Database access control determines what actions users are allowed to perform within a database environment. Depending on their roles, users may be permitted to:

- Read information
- Insert or modify records
- Delete data
- Manage database settings

Improper permission management can expose sensitive information and increase the risk of unauthorized activities.

Step 1: Reviewing Database User Roles and Permissions

The database structure was conceptually analyzed to understand how role-based permissions can be applied.

Typical Database Roles

- Database Administrator
- Application User
- Read-Only User
- Backup Operator

Risks of Excessive Privileges

When users are granted unnecessary permissions, attackers may be able to:

- Delete important records
- Modify sensitive information
- Access confidential data
- Execute administrative commands

Principle of Least Privilege

Description

Users should receive only the permissions required to perform their assigned tasks.

Example Permission Allocation

User Type	Required Permissions
Application User	SELECT, INSERT
Administrator	Full Administrative Access
Reporting User	SELECT Only

Recommended Access Control Practices

1. Remove unused database accounts.
2. Restrict administrative privileges.
3. Implement Role-Based Access Control (RBAC).
4. Review user permissions regularly.

Benefits

- Reduces insider threats.
- Prevents unauthorized modifications.
- Improves database security.

PART 2 – SECURING DATABASE AUTHENTICATION

Introduction

Weak authentication mechanisms can expose databases to unauthorized access and compromise sensitive information.

Step 2: Verifying Strong Password Usage

Strong passwords should be enforced for:

- Database administrator accounts
- Application database users
- Backup accounts

Recommended Password Policy

1. Minimum password length of 12 characters.
2. Combination of:
 - Uppercase letters
 - Lowercase letters
 - Numbers
 - Special symbols
3. Avoid dictionary words.
4. Change passwords periodically.

Examples of Strong Passwords

Example 1:

DB@Secure2026

Example 2:
Str0ng#DatabaseAccess

Multi-Factor Authentication (MFA)

Description

MFA adds an additional layer of security beyond traditional passwords.

Benefits

- 1. Prevents unauthorized access.
- 2. Reduces credential theft risks.
- 3. Improves administrator account security.

Common MFA Methods

MFA Method	Example
OTP Verification	SMS or Email Codes
Authentication Apps	Google Authenticator
Biometric Verification	Fingerprint or Face ID

Recommended Authentication Improvements

- 1. Enable MFA for administrator accounts.
- 2. Restrict remote database access.
- 3. Use secure VPN connections.
- 4. Disable default accounts.

PART 3 – PREVENTION OF SQL INJECTION

Introduction

SQL Injection is a common attack technique in which malicious input is used to manipulate database queries.

A successful SQL Injection attack may result in:

- Authentication bypass
- Unauthorized database access
- Data modification
- Data deletion

Step 3: Identifying SQL Query Risks

The website was conceptually analyzed to understand how insecure SQL queries can introduce vulnerabilities.

Example of an Insecure Query

```
SELECT * FROM users WHERE email = '$email'
```

Security Risk

Attackers can inject malicious input into query parameters.

Example Payload

```
' OR '1'='1
```

Step 4: Using Prepared Statements

Prepared statements separate user input from SQL commands and significantly reduce injection risks.

Secure Query Example

```
SELECT * FROM users WHERE email = ?
```

Benefits of Parameterized Queries

1. Prevent SQL Injection attacks.
2. Improve query security.
3. Separate user input from SQL logic.

Additional SQL Injection Prevention Techniques

1. Validate user inputs.
2. Sanitize dangerous characters.
3. Use stored procedures carefully.
4. Restrict database permissions.

PART 4 – ENCRYPTION OF SENSITIVE DATA

Introduction

Sensitive information stored in databases should be protected through encryption to prevent unauthorized access.

Examples of Sensitive Data

- Passwords
- Passport information
- Personal details
- Financial records

Step 5: Protecting Passwords

Passwords should never be stored in plain text format.

bcrypt Password Hashing

Description

bcrypt is a secure password hashing algorithm designed for protecting user credentials.

Example bcrypt Hash
\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

Argon2 Password Hashing

Description

Argon2 is a modern password hashing algorithm that provides strong resistance against brute-force attacks.

Benefits of Password Hashing

- 1. Protects credentials during database breaches.
- 2. Prevents plain-text password exposure.
- 3. Strengthens authentication security.

Encrypting Personal and Financial Data

Recommended Protection Methods

Data Type	Recommended Protection
Passwords	bcrypt / Argon2
Personal Information	AES Encryption
Financial Data	Strong Symmetric Encryption
Session Tokens	Secure Token Encryption

Benefits of Encryption

- Protects confidential information.
- Prevents unauthorized access.
- Minimizes the impact of data breaches.

PART 5 – DATABASE LOGGING AND BACKUPS

Introduction

Logging and backup mechanisms help organizations detect attacks, monitor activity, and recover data after failures.

Step 6: Enabling Database Logging

Database logs help track:

- Failed login attempts
- Suspicious queries
- Unauthorized access attempts
- Administrative changes

Example MySQL Logging Configuration

```
SET GLOBAL general_log = 'ON';
```

Benefits of Logging

1. Detects suspicious activity.
2. Supports incident investigation.
3. Monitors database access.

Log Analysis Tools

Tool	Purpose
Splunk	Security monitoring
ELK Stack	Centralized logging
Graylog	Log analysis
Fail2Ban	Blocks suspicious access

Step 7: Automated Database Backup

Importance of Backups

Backups help protect against:

- Data loss
- Ransomware attacks
- Hardware failures
- Human errors

Example MySQL Backup Command

```
mysqldump -u root -p database_name > backup.sql
```

Backup Recommendations

1. Schedule automated backups.
2. Store backups securely.
3. Encrypt backup files.
4. Regularly test backup restoration procedures.

Benefits of Automated Backups

- Faster recovery after failures.
- Reduced downtime.
- Improved disaster recovery readiness.

PART 6 – FINDINGS, FIXES, AND RECOMMENDATIONS

Security Findings

Issue Identified	Observation
Potential SQL Injection Risk	Direct query risks identified conceptually
Weak Authentication Controls	MFA not clearly visible
Sensitive Data Exposure Risk	Additional encryption recommended
Logging Improvements Needed	Advanced monitoring recommended
Backup Mechanism Not Visible	Automated backups recommended

Recommended Fixes

Security Issue	Recommended Fix
SQL Injection Risk	Use prepared statements
Weak Password Protection	Implement bcrypt or Argon2
Excessive User Privileges	Apply least privilege principle
Missing MFA	Enable Multi-Factor Authentication
Weak Monitoring	Implement centralized logging
Backup Risks	Configure automated backups

Additional Security Recommendations

1. Enable encryption for sensitive database information.
2. Restrict remote database access.
3. Use secure VPN connections for administrators.
4. Monitor database activity continuously.
5. Rotate administrator passwords regularly.
6. Remove unused database accounts.
7. Encrypt backup files.
8. Conduct regular database security audits.

Practical Observations During Testing

During testing and research activities:

1. HTTPS communication was observed.
2. POST requests transmitted data securely.
3. SQL Injection risks were studied conceptually.

4. Password hashing techniques such as bcrypt and Argon2 were researched.
5. Database logging and backup mechanisms were analyzed.

These observations highlighted the importance of implementing strong database security controls to protect sensitive user information.

Findings and Analysis

This practical demonstrated the critical role of database security in protecting Visa Processing and Consultancy systems.

Preventing SQL Injection through prepared statements and parameterized queries significantly reduces database attack risks. Strong authentication mechanisms, password hashing, data encryption, logging, and backup strategies further improve confidentiality, integrity, and availability.

Continuous monitoring, least-privilege access control, and automated backups are essential components of a secure database environment.

Conclusion

This practical successfully demonstrated database security analysis and protection techniques for a Visa Consultancy website.

The activity improved understanding of:

- Database access control
- SQL Injection prevention
- Secure authentication mechanisms
- Password hashing techniques
- Data encryption methods
- Database monitoring
- Backup and recovery strategies.

API Security Analysis and Protection of a Visa Consultancy Website

Aim

To strengthen the security of APIs used by a Visa Consultancy website by analyzing authentication mechanisms, validating API inputs, implementing rate limiting, securing data transmission, enabling API monitoring, and documenting security findings and recommendations.

Objectives

- To analyze API authentication mechanisms.
- To study secure API authentication using JWT.
- To understand API input validation techniques.
- To research and implement rate limiting concepts.
- To examine HTTPS encryption for secure API communication.
- To study API logging and monitoring practices.
- To document security findings, fixes, and recommendations.

Tools Used

Tool	Purpose
Burp Suite Community Edition	API request analysis
Browser Developer Tools	API inspection
JWT (JSON Web Token)	API authentication
HTTPS/TLS Protocol	Secure communication
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for API security analysis and cybersecurity research.

PART 1 – ANALYZING API AUTHENTICATION

Introduction

APIs are responsible for exchanging data between frontend applications and backend systems. Weak authentication mechanisms can allow attackers to gain unauthorized access to sensitive resources and user information.

Step 1: Reviewing API Authentication Mechanisms

The website was analyzed using Burp Suite and browser developer tools to inspect API communication and authentication controls.

Steps Performed

1. Opened the website using Burp Browser.
2. Accessed **Proxy** → **HTTP History** in Burp Suite.
3. Submitted sample form requests.
4. Inspected request headers and server responses.

Security Observations

Security Feature	Observation
HTTPS Enabled	Yes
JSON Communication	Observed
Authentication Tokens	Not Clearly Visible

Session Handling	Present
------------------	---------

Risks of Weak API Authentication

Weak authentication mechanisms may lead to:

- Unauthorized access
- Account compromise
- API abuse
- Data leakage

JWT (JSON Web Token) Authentication

Description

JWT is a token-based authentication mechanism commonly used to secure APIs.

A JWT consists of three components:

1. Header
2. Payload
3. Signature

Example JWT Structure

header.payload.signature

Benefits of JWT Authentication

1. Secure user authentication.
2. Stateless authentication process.
3. Reduced server-side session storage.
4. Improved API access control.

Example Authorization Header

Authorization: Bearer <JWT_TOKEN>

Recommended Authentication Improvements

1. Implement JWT authentication.
2. Configure token expiration policies.
3. Store authentication tokens securely.
4. Enable Multi-Factor Authentication (MFA).

PART 2 – VALIDATING API INPUTS

Introduction

Improper API input validation can expose applications to various security threats, including:

- SQL Injection
- Cross-Site Scripting (XSS)
- Malicious payload submissions
- API abuse

Step 2: Identifying API Endpoints Accepting User Input

Several API input sources were analyzed, including:

- Contact forms
- Inquiry forms
- Email fields
- Search parameters

Example API Request Payload

```
{  
  "name": "User",  
  "email": "user@example.com",  
  "message": "Visa Inquiry"  
}
```

Risks of Improper Validation

Attackers may attempt to submit:

- SQL Injection payloads
- Malicious JavaScript code
- Oversized requests
- Invalid data formats

SQL Injection Example

' OR '1'='1

XSS Example

<script>alert('XSS')</script>

Recommended Input Validation Techniques

Validation Technique	Purpose
Length Validation	Prevent oversized payloads
Email Validation	Accept valid email formats
Numeric Validation	Restrict numeric fields
Character Filtering	Block malicious input
Server-Side Validation	Prevent bypass attacks

Secure Validation Recommendations

1. Validate all API parameters.
2. Sanitize dangerous characters.
3. Reject invalid data formats.
4. Limit request sizes.

5. Use parameterized database queries.

Benefits of Input Validation

- Prevents injection attacks.
 - Improves API reliability.
 - Protects backend systems.
-

PART 3 – IMPLEMENTATION OF RATE LIMITING

Introduction

Rate limiting restricts the number of API requests that can be made within a specific time period.

It helps prevent:

- API abuse
- Brute-force attacks
- Denial-of-Service (DoS) attacks
- Excessive traffic

Step 3: Studying Rate Limiting Techniques

Common rate-limiting methods include:

- IP-based restrictions
- Token-based limits
- Request counters
- Time-window controls

Example Rate-Limit Policies

Limit Type	Example
Requests Per Minute	100
Login Attempts	5

API Requests Per Hour	1000
-----------------------	------

Example Nginx Configuration

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

Example Express.js Configuration

```
const rateLimit = require("express-rate-limit");
```

```
const limiter = rateLimit({  
  windowMs: 15 * 60 * 1000,  
  max: 100  
});
```

Benefits of Rate Limiting

1. Prevents API abuse.
2. Reduces brute-force attacks.
3. Protects servers from overload.
4. Improves service availability.

Recommended Practices

1. Limit authentication attempts.
2. Apply restrictions to sensitive endpoints.
3. Monitor excessive traffic.
4. Block suspicious IP addresses.

PART 4 – ENCRYPTING DATA IN TRANSIT

Introduction

Sensitive API communication should always be protected using encryption to prevent interception and unauthorized access.

Step 4: Verifying HTTPS Communication

The website was inspected to confirm secure communication practices.

Observation

HTTPS communication was enabled.

Security Headers Observed

Security Header	Observation
Strict-Transport-Security (HSTS)	Present
HTTPS	Enabled

Importance of HTTPS

HTTPS protects against:

- Credential theft
- Session hijacking
- Data interception
- Man-in-the-Middle attacks

Protecting Sensitive API Data

The following information should always be encrypted during transmission:

- Personal information
- Passport details
- Financial records
- Authentication tokens

Recommended Protection Methods

Data Type	Recommended Protection
API Communication	HTTPS/TLS
Passwords	bcrypt / Argon2
Sensitive Information	AES Encryption
Authentication Tokens	Secure JWT Signing

Benefits of Encryption

1. Protects confidential information.
2. Prevents unauthorized interception.
3. Improves communication security.

PART 5 – API LOGGING AND MONITORING

Introduction

API monitoring helps identify suspicious activities, security incidents, and unauthorized access attempts.

Step 5: Enabling API Logging

API logging helps track:

- Failed requests
- Unauthorized access attempts
- Excessive traffic
- Suspicious payloads

Example API Log Entry
POST /api/login - 401 Unauthorized

Recommended Logging Information

1. Request method
2. API endpoint
3. Timestamp
4. IP address
5. Response status code

API Monitoring Tools

Tool	Purpose
Splunk	Log analysis
ELK Stack	Centralized monitoring
Graylog	Security event analysis
Fail2Ban	Blocks suspicious IP addresses

Suspicious Behaviors Studied

The following suspicious activities were conceptually analyzed:

1. Repeated failed login attempts.
2. Excessive API requests.
3. Injection payload submissions.
4. Unauthorized access attempts.

Monitoring Recommendations

1. Enable centralized logging.
2. Monitor failed authentication attempts.
3. Detect abnormal request patterns.
4. Configure automated alerts.

Benefits of API Monitoring

- Early attack detection.
- Faster incident response.
- Improved security visibility.

PART 6 – FINDINGS, FIXES, AND RECOMMENDATIONS

Security Findings

Issue Identified	Observation
HTTPS Enabled	Yes
JWT Authentication	Not Clearly Visible
Input Validation	Additional validation recommended
Rate Limiting	Not Clearly Visible
API Monitoring	Advanced monitoring recommended
Injection Risks	Input sanitization required

Recommended Fixes

Security Issue	Recommended Fix
Weak Authentication	Implement JWT authentication
Injection Risks	Validate and sanitize inputs
Excessive API Requests	Configure rate limiting
Weak Monitoring	Enable API logging
Sensitive Data Exposure	Encrypt sensitive responses
Unauthorized Access Risk	Implement MFA

API Security Best Practices

1. Use HTTPS for all API communication.
2. Implement JWT or OAuth authentication.
3. Validate all user inputs.
4. Apply rate limiting to API endpoints.
5. Enable centralized API logging.
6. Encrypt sensitive information.
7. Restrict unauthorized access.
8. Continuously monitor API traffic.

Practical Observations During Testing

During practical testing and research activities:

1. HTTPS communication was enabled.
2. JSON-based API communication was observed.
3. HSTS security headers were present.
4. Authentication token mechanisms were not clearly visible.
5. Input validation and rate-limiting protections were not clearly observable.

These observations highlighted the importance of implementing layered API security controls to protect sensitive information and backend services.

Findings and Analysis

This practical demonstrated the importance of API security in protecting Visa Processing and Consultancy systems. Secure authentication, proper input validation, encryption, rate limiting, and monitoring are critical for maintaining the confidentiality, integrity, and availability of APIs.

JWT-based authentication improves access control, while input validation and rate limiting reduce the risks of injection attacks and API abuse. Continuous monitoring and HTTPS encryption further strengthen API security.

Conclusion

This practical successfully demonstrated API security analysis and protection techniques for a Visa Consultancy website.

The activity improved understanding of:

- API authentication mechanisms
- JWT security implementation
- Input validation techniques
- Rate limiting concepts
- HTTPS encryption
- API monitoring and logging
- Secure API development practices

Implementing strong API security controls is essential for protecting sensitive user information, preventing unauthorized access, and maintaining secure online services in modern web applications.

Basic Penetration Testing of a Visa Consultancy Website

Aim

To perform basic penetration testing on a Visa Consultancy website by identifying potential vulnerabilities related to authentication, SQL Injection, Cross-Site Scripting (XSS), session management, and file uploads using industry-standard security testing tools.

Objectives

- To set up a penetration testing environment.
- To perform automated vulnerability scanning.
- To test for SQL Injection vulnerabilities.
- To test for Cross-Site Scripting (XSS) vulnerabilities.
- To assess authentication and session management security.
- To analyze file upload security.
- To document vulnerabilities, risk levels, and recommended security fixes.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Request interception and security testing
OWASP ZAP	Automated vulnerability scanning
Mozilla Firefox / Burp Browser	Website interaction
SQL Injection Payloads	Database security testing
XSS Payloads	Script injection testing
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for penetration testing and cybersecurity analysis activities.

PART 1 – SETTING UP THE TESTING ENVIRONMENT

Introduction

A controlled testing environment is essential for conducting penetration testing safely and effectively. It allows security assessments to be performed without affecting production systems or user data.

Step 1: Installing Security Testing Tools

The following tools were installed and configured for testing:

1. Burp Suite Community Edition
2. OWASP ZAP
3. Mozilla Firefox / Burp Browser

Purpose of Tools

Tool	Function
Burp Suite	Intercept and analyze requests
OWASP ZAP	Perform automated vulnerability scanning
Browser Developer Tools	Inspect requests and response headers

Step 2: Configuring Burp Suite

Steps Performed

1. Opened Burp Suite.
2. Navigated to Proxy → Options.
3. Verified the proxy listener configuration:
 - 127.0.0.1:8080
4. Opened Burp Browser.
5. Accessed the target website.

Observation

Traffic interception was successfully enabled, allowing requests and responses to be analyzed during testing.

PART 2 – AUTOMATED VULNERABILITY SCANNING

Introduction

Automated vulnerability scanners help security professionals quickly identify common web application vulnerabilities and configuration weaknesses.

Step 3: Performing Vulnerability Scans

OWASP ZAP and Burp Suite were used to perform automated security analysis of the website.

Areas Analyzed

- Web pages
- Forms
- HTTP headers
- API requests
- Authentication mechanisms

Vulnerabilities Checked

Vulnerability Type	Purpose
Outdated Libraries	Detect insecure software
Weak Authentication	Identify authentication weaknesses
Missing Security Headers	Analyze browser security controls
SSL/TLS Issues	Evaluate communication security
Input Validation Weaknesses	Identify injection attack risks

Observations

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. Some advanced security headers were not clearly visible.
4. Backend timeout responses were occasionally observed.

Importance of Automated Scanning

Automated scanning helps:

- Detect common vulnerabilities.
- Improve assessment efficiency.
- Identify security misconfigurations.

PART 3 – SQL INJECTION TESTING

Introduction

SQL Injection is a vulnerability that occurs when user input is improperly handled within SQL queries, allowing attackers to manipulate database operations.

Step 4: Identifying Input Fields

The following input fields were analyzed:

- Contact forms
- Inquiry forms
- Search parameters
- Email fields

SQL Injection Payloads Tested

' OR '1'='1

admin' --

Testing Procedure

1. Inserted SQL payloads into available input fields.
2. Submitted requests through Burp Suite.
3. Monitored responses for:
 - SQL error messages
 - Authentication bypass
 - Unexpected behavior

Observation

No direct SQL Injection vulnerability was successfully exploited during testing.

However, implementing strong server-side validation and parameterized queries remains essential.

Risks of SQL Injection

1. Unauthorized database access.
2. Sensitive data exposure.
3. Authentication bypass.
4. Data modification or deletion.

Recommended Fixes

Security Improvement	Purpose
Prepared Statements	Prevent SQL Injection
Input Validation	Restrict malicious input
Parameterized Queries	Separate SQL logic from user input
Least Privilege Access	Restrict database permissions

Secure Query Example

```
SELECT * FROM users WHERE email = ?
```

PART 4 – CROSS-SITE SCRIPTING (XSS) TESTING

Introduction

Cross-Site Scripting (XSS) vulnerabilities occur when malicious scripts are executed within a user's browser due to improper handling of user input.

Step 5: Testing for XSS Vulnerabilities

Payloads Used

`<script>alert('XSS')</script>`

`<img src=x onerror=alert(1)>`

Testing Procedure

1. Entered payloads into available form fields.
2. Submitted requests.
3. Observed browser behavior for script execution.

Observation

The injected scripts were not executed during testing.

However, additional sanitization and output encoding mechanisms are recommended to further reduce XSS risks.

Risks of XSS

1. Session hijacking.
2. Cookie theft.
3. Credential theft.
4. Malicious redirection.

Recommended Fixes

Security Improvement	Purpose
Input Sanitization	Remove malicious code
Output Encoding	Prevent script execution

Content-Security-Policy	Reduce XSS risks
Character Filtering	Restrict dangerous input

PART 5 – AUTHENTICATION AND SESSION MANAGEMENT TESTING

Introduction

Weak authentication controls and insecure session management can allow attackers to gain unauthorized access to user accounts and protected resources.

Step 6: Authentication Analysis

The following areas were reviewed:

- Login mechanisms
- Session cookies
- Password handling
- Password reset functionality

Session Security Observations

Security Feature	Observation
HTTPS Enabled	Yes
Session Cookies	Present
MFA	Not Clearly Visible

CAPTCHA	Not Clearly Visible
Security Headers	Present

Session Hijacking Analysis

Session handling was analyzed conceptually using intercepted requests and response data.

Potential Risks

- Session hijacking
- Session fixation
- Weak session expiration controls

Weak Password Attack Analysis

The impact of weak passwords was studied conceptually.

Risks

- Brute-force attacks
- Credential stuffing
- Unauthorized account access

Recommended Authentication Improvements

1. Enable Multi-Factor Authentication (MFA).
2. Enforce strong password policies.
3. Implement account lockout mechanisms.
4. Use secure session cookies.
5. Regenerate session IDs after login.

PART 6 – FILE UPLOAD SECURITY TESTING

Introduction

Improperly secured file upload functionality may allow attackers to upload malicious files and compromise the server.

Step 7: File Upload Analysis

The website was examined for:

- File upload forms
- Document submission features
- Attachment upload functionality

Risks of Insecure File Uploads

1. Malware uploads.
2. Remote code execution.
3. Unauthorized file access.
4. Server compromise.

Security Checks Performed

Security Check	Purpose
File Type Validation	Restrict dangerous file formats
File Size Limits	Prevent abuse
File Renaming	Prevent path traversal attacks
Malware Scanning	Detect malicious uploads

Recommended Security Measures

1. Allow only approved file formats.
2. Rename uploaded files securely.
3. Store uploads outside public directories.
4. Scan files for malware.
5. Restrict executable file uploads.

Example File Types

Allowed Files

- .pdf
- .jpg
- .png

Blocked Files

- .php
- .exe
- .js

PART 7 – FINDINGS, RISK LEVELS, AND RECOMMENDATIONS

Security Findings

Vulnerability	Observation	Risk Level
SQL Injection Risk	No direct exploitation confirmed	Medium
XSS Risk	Additional sanitization recommended	Medium
Weak Authentication Protections	MFA not visible	Medium

Missing Advanced Security Headers	CSP/X-Frame-Options unclear	Medium
Potential File Upload Risks	Additional validation recommended	Medium
HTTPS Communication	Enabled	Low

Recommended Fixes

Security Issue	Recommended Fix
SQL Injection Risk	Use prepared statements
XSS Risk	Implement sanitization and CSP
Weak Authentication	Enable MFA and CAPTCHA
Session Security	Use secure cookies and expiration policies
File Upload Risks	Restrict file types and scan uploads
Weak Monitoring	Enable logging and alerts

Additional Security Recommendations

1. Implement Content-Security-Policy (CSP) headers.
2. Use bcrypt or Argon2 for password hashing.
3. Apply rate limiting on authentication endpoints.
4. Monitor suspicious traffic continuously.
5. Conduct regular penetration testing.
6. Keep server software updated.
7. Enable centralized logging and monitoring.

Practical Observations During Testing

During practical testing:

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. Input validation protections were partially visible.
4. No direct SQL Injection or XSS exploitation was confirmed.
5. Advanced authentication protections such as MFA were not clearly visible.

These observations highlight the importance of implementing multiple layers of security controls within modern web applications.

Findings and Analysis

This practical demonstrated the importance of penetration testing in identifying weaknesses within Visa Processing and Consultancy systems. Testing activities focused on SQL Injection, XSS vulnerabilities, authentication controls, session management, and file upload security.

Tools such as Burp Suite and OWASP ZAP helped analyze requests, identify potential security concerns, and understand existing protections implemented by the website.

Strong authentication, secure input validation, HTTPS encryption, secure file handling, and continuous monitoring are essential for reducing cybersecurity risks and protecting sensitive user information.

Conclusion

This practical successfully demonstrated basic penetration testing techniques for a Visa Consultancy website.

The activity improved understanding of:

- Vulnerability scanning
- SQL Injection testing
- XSS testing
- Authentication security
- Session management
- File upload security
- Security assessment methodologies

Implementing strong cybersecurity controls and conducting regular penetration testing are essential for protecting sensitive information and maintaining secure online services.

Incident Response Simulation for a Visa Consultancy Website

Aim

To simulate a cybersecurity incident response process for a Visa Consultancy website by identifying a security attack, containing the threat, investigating suspicious activities, eliminating vulnerabilities, recovering affected systems, and documenting the complete incident response lifecycle.

Objectives

- To understand common cybersecurity incidents affecting web applications.
- To learn threat containment and isolation procedures.
- To analyze logs and investigate suspicious activities.
- To study vulnerability remediation and threat eradication techniques.
- To understand recovery and backup restoration procedures.
- To document the incident response process and lessons learned.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Traffic and request analysis
Browser Developer Tools	Network inspection
Log Monitoring Tools	Incident investigation
Backup and Recovery Concepts	System restoration
Visa Consultancy Website	Security incident simulation target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for incident response simulations and cybersecurity analysis.

PART 1 – IDENTIFYING A SECURITY INCIDENT

Introduction

Web applications are frequently targeted by cyberattacks that can compromise sensitive information, user accounts, databases, and server infrastructure.

Common cybersecurity threats include:

- Malware attacks
- Phishing attacks
- SQL Injection
- Cross-Site Scripting (XSS)
- Credential theft
- Data breaches

Proper incident identification is the first step toward minimizing damage and restoring secure operations.

Selected Incident Scenario – SQL Injection Attack

Description

A simulated SQL Injection attack scenario was chosen for this practical activity.

In this scenario, an attacker attempts to exploit vulnerable user input fields within the Visa Consultancy website to gain unauthorized access to the backend database.

Example Malicious Payload

' OR '1'='1

Potential Impact on the Website

Impact Area	Possible Consequence
User Accounts	Unauthorized access
Personal Information	Data leakage
Database	Record modification or deletion
Authentication System	Login bypass
Business Operations	Service disruption

Risks Associated with the Incident

1. Exposure of applicant information.
2. Unauthorized database access.
3. Data modification or deletion.
4. Damage to organizational reputation.
5. Legal and compliance issues.

Importance of Incident Response

A structured incident response process helps organizations:

- Minimize damage.
- Prevent attack escalation.
- Restore services quickly.
- Protect sensitive information.

PART 2 – CONTAINING THE THREAT

Introduction

Threat containment focuses on isolating affected systems and preventing attackers from causing additional damage.

Step 1: Isolating Affected Systems

The affected application components were conceptually isolated to prevent further exploitation.

Actions Performed

1. Restricted suspicious IP addresses.
2. Disabled potentially vulnerable application endpoints.
3. Temporarily limited database access.
4. Monitored suspicious network traffic.

Step 2: Blocking Malicious Requests

Firewall and Web Application Firewall (WAF) rules were reviewed and updated to block:

- Malicious payloads
- SQL Injection attempts
- Suspicious IP addresses

Example Firewall Action

Block access from identified suspicious IP addresses.

Step 3: Disabling Potentially Compromised Accounts

If unauthorized access is suspected:

1. Reset account passwords.
2. Revoke active sessions.
3. Enable Multi-Factor Authentication (MFA).

Methods Used to Prevent Attack Spread

Prevention Method	Purpose
Network Isolation	Stop attack propagation
Firewall Rules	Block malicious traffic
Session Revocation	Remove unauthorized access
WAF Protection	Filter malicious requests

Importance of Threat Containment

Containment helps:

- Reduce attack impact.
- Protect unaffected systems.
- Prevent data loss.

PART 3 – INVESTIGATING THE INCIDENT

Introduction

Incident investigation helps determine how an attack occurred, which systems were affected, and what information may have been exposed.

Step 4: Log Analysis

System and application logs were conceptually reviewed to identify suspicious activities.

Logs Examined

Log Type	Purpose
Access Logs	Track incoming requests
Error Logs	Detect application issues
Authentication Logs	Identify failed logins
Database Logs	Detect suspicious queries

Example Suspicious Log Entry

POST /login ' OR '1'='1

Indicators of Suspicious Activity

1. Repeated failed login attempts.
2. Unusual SQL query patterns.
3. Unexpected traffic spikes.
4. Unauthorized API requests.

Identifying the Entry Point

Suspected Entry Point

User input fields lacking proper validation and sanitization controls.

Nature of the Attack

Attack Type: SQL Injection

Attack Goal: Unauthorized database access

Importance of Investigation

Investigation helps:

- Understand attacker behavior.
- Identify exploited vulnerabilities.
- Improve future defenses.

PART 4 – ERADICATING THE THREAT

Introduction

Threat eradication focuses on removing malicious activity and eliminating vulnerabilities that allowed the attack to occur.

Step 5: Removing the Vulnerability

The vulnerable input handling mechanism was conceptually secured using:

- Input validation
- Prepared statements
- Parameterized queries

Secure SQL Query Example

SELECT * FROM users WHERE email = ?

Step 6: Applying Security Improvements

The following corrective actions were recommended:

1. Update web server software.
2. Patch outdated libraries.

3. Enable additional security headers.
4. Improve authentication mechanisms.

Malware Removal Research

If malware is detected:

1. Scan affected systems.
2. Remove malicious files.
3. Restore trusted application versions.
4. Verify file integrity after restoration.

Recommended Security Enhancements

Security Improvement	Purpose
Prepared Statements	Prevent SQL Injection
Security Updates	Patch vulnerabilities
Input Validation	Restrict malicious input
MFA	Strengthen authentication
WAF	Block malicious traffic

Importance of Threat Eradication

Eradication helps:

- Remove malicious activity.
- Prevent reinfection.
- Strengthen application security.

PART 5 – RECOVERING THE SYSTEM

Introduction

Recovery focuses on restoring affected systems and services after vulnerabilities have been addressed.

Step 7: Restoring Systems Using Backups

Backups play a critical role in recovering:

- Databases
- Website files
- System configurations

Backup Recovery Process

1. Verify backup integrity.
2. Restore affected systems.
3. Reconfigure security settings.
4. Test application functionality.

Security Checks Before Recovery

Before bringing systems back online:

1. Confirm vulnerabilities have been fixed.
2. Reset compromised credentials.
3. Enable logging and monitoring.
4. Verify firewall protection.

Recovery Recommendations

Recovery Action	Purpose
Restore Clean Backups	Recover lost data
Re-enable Services Safely	Prevent reinfection
Verify Security Controls	Improve protection
Monitor Traffic	Detect recurring attacks

Importance of Recovery

Recovery ensures:

- Business continuity.
- Secure restoration of services.
- Reduced operational downtime.

PART 6 – INCIDENT REPORT AND LESSONS LEARNED

Incident Summary

A simulated SQL Injection attack scenario was analyzed for the Visa Consultancy website.

The attack targeted vulnerable input fields and attempted unauthorized access to database resources through malicious SQL payloads.

Incident Timeline

Phase	Activity
Detection	Suspicious SQL payload identified
Containment	Restricted malicious traffic
Investigation	Analyzed logs and requests
Eradication	Improved input validation
Recovery	Planned secure restoration

Vulnerabilities Identified

Vulnerability	Risk Level
Weak Input Validation	High
SQL Injection Risk	High
Missing MFA	Medium
Weak Monitoring	Medium

Recommended Security Fixes

Security Fix	Purpose
Prepared Statements	Prevent SQL Injection
Input Sanitization	Restrict malicious input
Logging and Monitoring	Detect attacks
MFA	Improve authentication
Security Updates	Patch vulnerabilities

Lessons Learned

1. Proper input validation is critical for preventing SQL Injection attacks.
2. Continuous monitoring improves threat detection capabilities.
3. Strong authentication mechanisms reduce unauthorized access risks.
4. Reliable backups are essential for rapid recovery.
5. Incident response planning improves organizational readiness.

Additional Security Recommendations

1. Conduct regular penetration testing.
2. Implement a Web Application Firewall (WAF).
3. Enable centralized logging.
4. Enforce Multi-Factor Authentication.
5. Apply software updates regularly.
6. Monitor suspicious traffic continuously.
7. Conduct cybersecurity awareness training.

Practical Observations During Simulation

During the incident response simulation:

1. SQL Injection risks were analyzed conceptually.
2. HTTPS communication was observed.
3. HSTS security headers were present.
4. Input validation improvements were recommended.
5. Logging and monitoring mechanisms were studied.

These observations highlighted the importance of rapid response procedures and layered security controls for protecting web applications.

Findings and Analysis

This practical demonstrated the importance of incident response procedures in protecting Visa Processing and Consultancy systems from cyber threats.

The activities covered all major stages of incident response, including detection, containment, investigation, eradication, and recovery. Security controls such as prepared statements, logging, strong authentication, backups, and continuous monitoring were identified as essential measures for improving cybersecurity resilience.

A well-planned incident response strategy significantly reduces the impact of attacks and helps organizations restore operations quickly and securely.

Conclusion

This practical successfully demonstrated an incident response simulation for a Visa Consultancy website.

The activity improved understanding of:

- Cyberattack identification
- Threat containment techniques
- Log analysis and investigation
- Vulnerability remediation
- Backup and recovery procedures
- Security monitoring
- Incident response documentation

Implementing strong incident response planning and layered cybersecurity controls is essential for protecting sensitive user information and maintaining secure online services.

Implementation of Secure Coding Practices for a Visa Consultancy Website

Aim

To implement secure coding practices for a Visa Consultancy website by identifying security weaknesses in application code, validating user input, strengthening authentication systems, encrypting sensitive information, removing insecure dependencies, and documenting security improvements.

Objectives

- To identify security issues in existing application code.
- To implement secure input validation techniques.
- To strengthen authentication and authorization mechanisms.
- To encrypt sensitive user information.
- To remove insecure and outdated dependencies.
- To document secure coding improvements and code modifications.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Security testing
Browser Developer Tools	Request inspection
bcrypt / Argon2	Password hashing
npm audit / pip-audit	Dependency vulnerability scanning
Express Validator / Django Forms	Input validation
Visa Consultancy Website	Security analysis target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for secure coding analysis and cybersecurity research.

PART 1 – REVIEWING EXISTING CODE FOR SECURITY ISSUES

Introduction

Insecure coding practices can expose web applications to various security threats and vulnerabilities. Common risks include:

- SQL Injection
- Cross-Site Scripting (XSS)
- Credential theft
- Unauthorized access
- Data breaches

The application code and website functionality were conceptually reviewed to identify areas that could be improved from a security perspective.

Step 1: Identifying Hardcoded Credentials and API Keys

Description

Hardcoded credentials stored directly within source code can expose sensitive information such as:

- Database passwords
- API keys
- Administrator credentials

Example of Insecure Code

```
const dbPassword = "admin123";  
const apiKey = "ABC123SECRET";
```

Security Risks

1. Credential exposure.
2. Unauthorized system access.
3. API misuse.
4. Data compromise.

Secure Approach

Sensitive values should be stored securely using:

- Environment variables
- Secret management solutions

Secure Example

```
const dbPassword = process.env.DB_PASSWORD;  
const apiKey = process.env.API_KEY;
```

Step 2: Identifying Weak Input Validation

The following user input areas were analyzed:

- Login forms
- Contact forms
- Search fields
- Registration forms

Risks of Improper Validation

Poor input validation may lead to:

- SQL Injection attacks
- XSS vulnerabilities
- Execution of malicious payloads

Observation

Additional server-side validation and sanitization mechanisms were recommended to improve security.

PART 2 – IMPLEMENTING SECURE INPUT VALIDATION

Introduction

Input validation ensures that only properly formatted and expected data is accepted by the application.

This reduces the risk of malicious input and improves application reliability.

Step 3: Using Validation Libraries

Node.js Example – Express Validator

```
const { body, validationResult } = require('express-validator');
```

```
app.post('/login',  
[  
  body('email').isEmail(),  
  body('password').isLength({ min: 8 })  
],  
(req, res) => {  
  const errors = validationResult(req);
```

```
if (!errors.isEmpty()) {  
    return res.status(400).json({ errors: errors.array() });  
}  
});
```

Python Example – Django Forms

```
from django import forms
```

```
class LoginForm(forms.Form):  
    email = forms.EmailField()  
    password = forms.CharField(min_length=8)
```

Benefits of Validation Libraries

1. Validate input formats.
2. Restrict malicious input.
3. Improve application reliability.
4. Prevent injection attacks.

Step 4: Sanitizing Special Characters

Example XSS Payload

```
<script>alert('XSS')</script>
```

Risks of Unsanitized Input

1. Cross-Site Scripting attacks.
2. Session hijacking.
3. Credential theft.

Recommended Sanitization Techniques

Technique	Purpose
Character Escaping	Prevent script execution
Input Sanitization	Remove malicious content
Output Encoding	Prevent browser interpretation
Length Validation	Restrict oversized input

Importance of Input Validation

Secure validation protects applications against:

- SQL Injection
- Cross-Site Scripting (XSS)
- Malicious payload execution

PART 3 – SECURING AUTHENTICATION AND AUTHORIZATION

Introduction

Weak authentication and authorization controls increase the likelihood of unauthorized access and account compromise.

Strong access control mechanisms help protect sensitive application resources.

Step 5: Implementing Role-Based Access Control (RBAC)

Description

RBAC restricts access to system resources based on user roles and responsibilities.

Example Roles

Role	Permissions
Admin	Full system access
Staff	Limited management access
User	Application access only

Benefits of RBAC

1. Restricts unauthorized actions.
2. Reduces privilege abuse.
3. Simplifies access management.

Step 6: Implementing Secure Password Hashing

Passwords should never be stored in plain text format.

Example of Insecure Password Storage

```
password = "user123";
```

Secure bcrypt Example

```
const bcrypt = require('bcrypt');
```

```
const hashedPassword = await bcrypt.hash(password, 12);
```

Example bcrypt Hash

\$2b\$12\$8Fh2K1n4fB3DkT9sQxL2OeXvM7rZP4YjA6nKpW1uV5hQ8eC0dL9uS

Argon2 Password Hashing

Description

Argon2 is a modern password hashing algorithm designed to resist brute-force attacks and password cracking techniques.

Benefits of Secure Password Hashing

1. Protects credentials during database breaches.
2. Prevents plain-text password exposure.
3. Strengthens authentication security.

Recommended Authentication Improvements

1. Enable Multi-Factor Authentication (MFA).
2. Enforce strong password policies.
3. Implement session expiration.
4. Regenerate session IDs after login.

PART 4 – ENCRYPTING SENSITIVE DATA

Introduction

Sensitive information should be protected both while stored and during transmission.

Sensitive Information Includes

- Passwords
- Financial information
- Personal details
- Passport information

Step 7: Encrypting Sensitive Information

Recommended Encryption Methods

Data Type	Recommended Protection
Passwords	bcrypt / Argon2
Sensitive Data	AES Encryption
Communication	HTTPS/TLS
API Tokens	Secure Token Signing

HTTPS Communication Analysis

Observation

HTTPS communication was enabled on the Visa Consultancy website.

Benefits of HTTPS

1. Prevents credential theft.
2. Protects transmitted data.
3. Prevents Man-in-the-Middle attacks.

Example Secure URL

<https://veerconsultancyglobal.com>

PART 5 – REMOVING UNUSED AND INSECURE DEPENDENCIES

Introduction

Outdated or vulnerable dependencies may expose applications to publicly known security vulnerabilities.

Regular dependency auditing helps maintain a secure software environment.

Step 8: Running Dependency Security Checks

Node.js Example

`npm audit`

Python Example

`pip-audit`

Purpose of Dependency Checks

Dependency scanning helps identify:

- Vulnerable packages
- Outdated libraries
- Known security issues

Updating Vulnerable Dependencies

Node.js Example

`npm update`

Python Example

`pip install --upgrade package_name`

Risks of Outdated Dependencies

1. Remote code execution vulnerabilities.
2. Authentication bypass issues.
3. Data leakage.
4. Server compromise.

Recommended Dependency Security Practices

1. Remove unused packages.
2. Update libraries regularly.
3. Monitor vulnerability advisories.
4. Use trusted and maintained packages only.

PART 6 – DOCUMENTING SECURITY IMPROVEMENTS

Security Improvements Implemented

Security Improvement	Purpose
Environment Variables	Protect credentials
Input Validation	Prevent injection attacks
Output Sanitization	Prevent XSS
bcrypt / Argon2 Hashing	Secure passwords
RBAC	Restrict access
HTTPS	Secure communication

Dependency Updates	Patch vulnerabilities
-----------------------	-----------------------

Before and After Secure Coding Examples

Example 1 – Hardcoded Credential Removal

Before

```
const password = "admin123";
```

After

```
const password = process.env.DB_PASSWORD;
```

Example 2 – SQL Injection Prevention

Before

```
SELECT * FROM users WHERE email = '$email'
```

After

```
SELECT * FROM users WHERE email = ?
```

Example 3 – Password Security

Before

```
password = "user123";
```

After

```
const hashedPassword = await bcrypt.hash(password, 12);
```

Security Findings

Security Issue	Observation
Hardcoded Credentials	Security risk identified conceptually
Weak Input Validation	Additional validation recommended
SQL Injection Risks	Parameterized queries required
Weak Password Storage	bcrypt / Argon2 recommended
Outdated Dependencies	Regular updates recommended

Additional Security Recommendations

1. Conduct regular code reviews.
2. Enable centralized logging.
3. Implement Multi-Factor Authentication.
4. Use secure session management.
5. Perform dependency audits regularly.
6. Apply Content-Security-Policy headers.
7. Use HTTPS for all communication.

Practical Observations During Testing

During practical testing and research activities:

1. HTTPS communication was enabled.
2. HSTS security headers were observed.
3. Secure password hashing techniques were researched.

4. Dependency auditing tools were studied.
5. Input validation and sanitization improvements were analyzed.

These observations highlighted the importance of secure coding practices in protecting sensitive user information and reducing cybersecurity risks.

Findings and Analysis

This practical demonstrated the importance of secure coding practices in protecting Visa Processing and Consultancy systems. Input validation, secure authentication, password hashing, encryption, dependency management, and Role-Based Access Control are essential for maintaining application security.

Implementing bcrypt or Argon2 hashing, parameterized queries, HTTPS encryption, secure dependency management, and proper access controls significantly improves application security and reduces the likelihood of cyberattacks.

Conclusion

This practical successfully demonstrated the implementation of secure coding practices for a Visa Consultancy website.

The activity improved understanding of:

- Secure coding principles
- Input validation techniques
- Authentication security
- RBAC implementation
- Password hashing
- Dependency auditing
- Secure data encryption

Strong secure coding practices are essential for protecting sensitive user information, preventing cyber threats, and maintaining secure online services.

Installation and Exploration of Burp Suite for a Visa Consultancy Website

Aim

To install and explore Burp Suite for analyzing the security of a Visa Consultancy website by intercepting HTTP requests, identifying sensitive data transmission, performing basic vulnerability assessments, and documenting security findings.

Objectives

- To install and configure Burp Suite.
- To set up proxy interception for web traffic analysis.
- To intercept and analyze HTTP/HTTPS requests.
- To perform a basic security assessment.
- To identify security weaknesses within the web application.
- To document findings and recommend security improvements.

Tools Used

Tool	Purpose
Burp Suite Community Edition	Web security testing
Burp Browser	Request interception
Mozilla Firefox	Proxy configuration
Browser Developer Tools	Request inspection
Visa Consultancy Website	Security testing target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for security testing and HTTP request analysis.

PART 1 – INSTALLATION AND LAUNCH OF BURP SUITE

Introduction

Burp Suite is a widely used web application security testing tool designed to help security professionals analyze and assess website security.

It can be used for:

- Intercepting HTTP and HTTPS traffic
- Analyzing requests and responses
- Identifying security vulnerabilities
- Performing penetration testing

Step 1: Downloading Burp Suite

Burp Suite Community Edition was downloaded from the official PortSwigger website and prepared for installation.

Purpose

To obtain the necessary software for web security testing and traffic analysis.

Step 2: Installing Burp Suite

Installation Process

1. Opened the installation package.
2. Selected the installation directory.

3. Completed the setup process.
4. Launched Burp Suite successfully.

Observation

The installation was completed without errors, and the application launched successfully.

Step 3: Creating a Temporary Project

Steps Performed

1. Opened Burp Suite.
2. Selected **Temporary Project**.
3. Clicked **Next**.
4. Chose **Use Burp Defaults**.
5. Clicked **Start Burp**.

Purpose

A temporary project was used to perform testing without storing project data permanently.

Step 4: Verifying the Proxy Listener

Burp Suite's proxy listener configuration was reviewed.

Steps Performed

1. Opened **Proxy** → **Options**.
2. Verified the proxy listener configuration:

127.0.0.1:8080

Observation

The proxy listener was active and functioning correctly.

Step 5: Opening Burp Browser

Steps Performed

1. Opened **Proxy** → **Intercept**.
2. Clicked **Open Browser**.

Purpose

The built-in Burp Browser was used to route website traffic through Burp Suite for analysis.

Importance of Burp Suite

Burp Suite helps security analysts:

- Monitor web traffic
- Identify vulnerabilities
- Test application security
- Perform penetration testing

PART 2 – INTERCEPTING AND ANALYZING REQUESTS

Introduction

Burp Suite allows interception of HTTP and HTTPS requests exchanged between the browser and the web server.

This capability helps security professionals inspect data transmission and identify potential weaknesses.

Step 6: Enabling Request Interception

Steps Performed

1. Opened **Proxy** → **Intercept**.
2. Enabled **Intercept is ON**.

Purpose

To capture requests before they reach the server for inspection and analysis.

Step 7: Accessing the Website

Website Tested

<https://veerconsultancyglobal.com>

Observation

HTTP and HTTPS requests began appearing within Burp Suite's interception interface.

Step 8: Capturing and Reviewing Requests

The following request components were examined:

- Request method
- URL
- HTTP headers
- Cookies
- Form data

Example HTTP Request

POST /contact HTTP/1.1

Host: veerconsultancyglobal.com

Content-Type: application/x-www-form-urlencoded

Sensitive Data Analysis

The following types of information were checked during testing:

- Login credentials
- Session cookies
- Email addresses
- Contact information

Security Observations

Security Feature	Observation
HTTPS Enabled	Yes
Session Cookies	Present
POST Requests	Used
HSTS Header	Present

Risks of Sensitive Data Exposure

If HTTPS is not implemented correctly:

- Credentials may be intercepted.
- Session hijacking may occur.
- Sensitive information may be exposed.

Importance of Request Analysis

Request analysis helps:

- Detect insecure communication.
- Identify authentication weaknesses.
- Analyze API behavior.
- Verify secure data transmission.

PART 3 – PERFORMING A BASIC SECURITY SCAN

Introduction

Burp Suite provides tools that assist in identifying common web application vulnerabilities and security weaknesses.

Step 9: Using HTTP History

Steps Performed

1. Opened **Proxy** → **HTTP History**.
2. Reviewed captured requests and responses.
3. Examined headers and parameters.

Purpose

To understand how the application processes requests and responses.

Vulnerabilities Checked

Vulnerability Type	Purpose
Missing Security Headers	Browser protection analysis
Weak Authentication	Login security assessment
Input Validation Issues	Injection attack detection
HTTPS Misconfiguration	Communication security review

Step 10: Testing Input Fields

The following input areas were reviewed:

- Contact forms
- Email fields
- Search fields

SQL Injection Payload Tested

' OR '1'='1

XSS Payload Tested

<script>alert('XSS')</script>

Observations

1. HTTPS communication was enabled.
2. HSTS security headers were present.
3. No direct SQL Injection vulnerability was confirmed.
4. Additional input validation and sanitization were recommended.

Common Security Weaknesses Identified

Weakness	Risk Level
Missing Advanced Security Headers	Medium
Potential Input Validation Weaknesses	Medium
MFA Not Clearly Visible	Medium
File Upload Protections Not Verified	Medium

Recommended Security Improvements

1. Implement Content-Security-Policy (CSP) headers.
2. Use parameterized SQL queries.
3. Enable Multi-Factor Authentication (MFA).
4. Apply strict input validation.
5. Use secure session management.

PART 4 – SECURITY REPORT AND DOCUMENTATION

Introduction

Security reports provide a structured record of vulnerabilities, findings, risks, and recommended remediation measures.

Step 11: Capturing Security Evidence

The following screenshots were identified as useful documentation:

1. Burp Suite Dashboard
2. Intercepted HTTP Requests
3. HTTP History Tab
4. Request Headers
5. Security Testing Activities

Security Findings

Vulnerability	Observation	Risk Level
HTTPS Enabled	Communication secured	Low
HSTS Header Present	Browser HTTPS enforcement	Low
Input Validation Improvements Needed	Potential injection risks	Medium
Missing Advanced Security Headers	Additional protection required	Medium
MFA Not Clearly Visible	Authentication risk	Medium

Suggested Fixes

Security Issue	Recommended Fix
Weak Input Validation	Validate and sanitize inputs
Missing CSP Header	Implement Content-Security-Policy
Weak Authentication	Enable MFA
SQL Injection Risks	Use prepared statements
Session Security	Use secure cookies

Additional Security Recommendations

1. Conduct regular penetration testing.
2. Enable centralized logging and monitoring.
3. Keep server software updated.
4. Apply rate limiting to authentication forms.
5. Use secure password hashing algorithms.

Practical Observations During Testing

During practical testing using Burp Suite:

1. HTTPS communication was active.
2. POST requests transmitted form data securely.
3. Session cookies were observed.
4. HSTS security headers were present.
5. No direct SQL Injection exploitation was confirmed.

These observations highlighted the importance of layered security controls and continuous security testing.

Findings and Analysis

This practical demonstrated how Burp Suite can be used for web application security testing and HTTP traffic analysis. Intercepting requests provided valuable insight into application behavior, security headers, authentication mechanisms, and data transmission practices.

The activity emphasized the importance of:

- HTTPS communication
- Input validation
- Security headers

- Secure authentication
- Session protection

Burp Suite proved to be an effective tool for identifying potential security weaknesses and improving cybersecurity awareness.

Conclusion

This practical successfully demonstrated the installation and exploration of Burp Suite for security testing of a Visa Consultancy website.

The activity improved understanding of:

- Burp Suite installation
- Proxy configuration
- Request interception
- HTTP/HTTPS analysis
- Vulnerability testing
- Security reporting

Regular security testing using tools such as Burp Suite is essential for protecting sensitive user information and maintaining secure web applications.

Security Monitoring Using Splunk for a Visa Consultancy Website

Aim

To install and configure Splunk for security monitoring, collect and analyze security logs from multiple sources, detect suspicious activities using SPL queries, create a security dashboard, and document security findings for a Visa Consultancy website.

Objectives

- To install and configure Splunk Enterprise.
- To collect and index security logs from different sources.
- To search for suspicious activities using SPL queries.
- To create a real-time security monitoring dashboard.
- To analyze potential security threats and mitigation techniques.
- To document security findings and recommendations.

Tools Used

Tool	Purpose
Splunk Enterprise	Security monitoring and log analysis
Windows/Linux System Logs	Event monitoring
Firewall Logs	Network security monitoring
Browser Developer Tools	Traffic inspection
Visa Consultancy Website	Security monitoring target

Target Website

Website: <https://veerconsultancyglobal.com>

The website was selected because it represents a Visa Processing and Consultancy platform suitable for cybersecurity monitoring and log analysis activities.

PART 1 – INSTALLATION AND CONFIGURATION OF SPLUNK

Introduction

Splunk is a Security Information and Event Management (SIEM) platform used for collecting, analyzing, and monitoring security-related data from multiple sources.

It helps organizations:

- Monitor security events
- Detect cyberattacks
- Analyze logs
- Generate alerts
- Investigate incidents

Step 1: Downloading Splunk Enterprise

Splunk Enterprise was downloaded from the official Splunk website and prepared for installation.

Purpose

To set up a centralized platform for security monitoring and log analysis.

Step 2: Installing Splunk Enterprise

Installation Steps

1. Opened the Splunk installer.
2. Accepted the license agreement.
3. Selected the installation location.
4. Created administrator credentials.
5. Completed the installation process.

Observation

The installation was completed successfully without any errors.

Step 3: Launching Splunk

Steps Performed

1. Opened Splunk Enterprise.
2. Accessed the web interface using:

`http://localhost:8000`

3. Logged in using administrator credentials.

Observation

The Splunk dashboard loaded successfully and was ready for configuration.

Importance of Splunk

Splunk helps security analysts:

- Centralize log collection
- Monitor security events
- Detect suspicious activities

- Investigate incidents
- Generate automated alert

PART 2 – COLLECTING AND INDEXING SECURITY LOGS

Introduction

Logs provide valuable information about application behavior, authentication events, network traffic, and system activities.

Proper log collection improves visibility into potential security threats.

Step 4: Adding Data Sources

The following log sources were configured for monitoring:

Data Source	Purpose
System Logs	Monitor operating system activity
Firewall Logs	Detect network attacks
Application Logs	Monitor web application behavior
Authentication Logs	Detect login failures

Steps to Add Logs in Splunk

1. Opened **Settings** → **Add Data**.
2. Selected the appropriate log source.
3. Uploaded or monitored log files.
4. Indexed logs into Splunk.

Example Log Sources

Log File	Purpose
access.log	Monitor web requests
error.log	Track application errors
firewall.log	Monitor network activity
auth.log	Analyze login events

Benefits of Log Collection

1. Detect suspicious behavior.
2. Monitor user activity.
3. Investigate incidents.
4. Improve threat visibility.

PART 3 – SEARCHING FOR SECURITY THREATS

Introduction

Splunk Processing Language (SPL) is used to search, filter, and analyze security logs.

It enables analysts to quickly identify suspicious activities and security incidents.

Step 5: Searching for Failed Login Attempts

Example SPL Query

```
index=main "failed login"
```

Purpose

This query searches logs for failed authentication attempts that may indicate unauthorized access attempts.

Step 6: Detecting Brute-Force Attacks

Example SPL Query

```
index=main "failed login"
```

```
| stats count by src_ip
```

```
| where count > 5
```

Purpose

This query identifies IP addresses generating multiple failed login attempts within a short period.

Indicators of Brute-Force Attacks

1. Multiple failed logins from the same IP address.
2. Rapid authentication attempts.
3. Unusual login activity.
4. Repeated password failures.

Example Security Threats Analyzed

Threat Type	Description
Brute-Force Attack	Repeated password guessing attempts
Unauthorized Access	Invalid login attempts
Suspicious Traffic	Unusual request activity
Web Attacks	Malicious payload submissions

Importance of Threat Detection

Threat detection helps organizations:

- Identify attacks early.
- Prevent unauthorized access.
- Improve incident response.
- Protect sensitive information.

PART 4 – CREATING A SECURITY DASHBOARD

Introduction

Security dashboards provide visual representations of security events and system activities in real time.

They help analysts quickly identify abnormal behavior and respond to threats.

Step 7: Creating a Security Dashboard

Steps Performed

1. Opened **Dashboards** → **Create New Dashboard**.
2. Entered a dashboard name.
3. Added visualizations using SPL queries.

Purpose

To provide a centralized view of important security events.

Dashboard Components

Dashboard Widget	Purpose
Failed Login Attempts	Detect brute-force attacks
Firewall Events	Monitor blocked traffic
HTTP Requests	Monitor website activity
Error Logs	Detect application issues

Example Dashboard Query

```
index=main  
| stats count by sourcetype
```

Purpose

Displays event counts grouped by log source type.

Visualization Types Used

1. Bar Charts
2. Pie Charts
3. Event Tables
4. Real-Time Event Panels

Benefits of Security Dashboards

1. Real-time monitoring.
2. Faster threat detection.
3. Improved visibility.
4. Easier incident investigation.

PART 5 – SECURITY ANALYSIS AND REPORTING

Introduction

Security reports help summarize observed threats, vulnerabilities, mitigation techniques, and recommended improvements.

Security Findings

Security Issue	Observation	Risk Level
Failed Login Attempts	Repeated authentication failures observed conceptually	Medium
Brute-Force Risk	Excessive login attempts possible	Medium

Monitoring Enabled	Logs collected successfully	Low
Suspicious Traffic Analysis	Threat monitoring performed conceptually	Medium

Recommended Mitigation Measures

Security Threat	Recommended Mitigation
Brute-Force Attacks	Implement account lockout policies
Unauthorized Access	Enable Multi-Factor Authentication
Suspicious Traffic	Configure firewall rules
Excessive Requests	Apply rate limiting
Weak Monitoring	Enable real-time alerts

Recommended Security Improvements

1. Enable centralized logging.
2. Configure real-time security alerts.
3. Implement Multi-Factor Authentication (MFA).
4. Apply rate limiting to login endpoints.
5. Continuously monitor suspicious IP addresses.
6. Conduct regular security audits.
7. Integrate firewall and application logs into Splunk.

Example Alert Query

```
index=main "failed login"  
| stats count by src_ip  
| where count > 10
```

Purpose

Generates alerts when excessive failed login attempts are detected from a single IP address.

Practical Observations During Monitoring

During practical monitoring activities:

1. Log ingestion was configured successfully.
2. SPL queries were used to analyze login failures.
3. Brute-force detection techniques were researched.
4. Dashboard visualizations were created conceptually.
5. Threat monitoring workflows were studied.

These observations highlighted the importance of SIEM tools in cybersecurity monitoring and incident detection.

Findings and Analysis

This practical demonstrated the importance of security monitoring in protecting Visa Processing and Consultancy systems. Splunk provides centralized visibility into authentication logs, firewall events, application activity, and security incidents.

SPL queries help identify suspicious behavior such as brute-force attacks, repeated authentication failures, and unauthorized access attempts. Security dashboards improve operational visibility and support faster incident response.

Real-time monitoring, centralized logging, and alerting mechanisms are essential for maintaining strong cybersecurity defenses.

Conclusion

This practical successfully demonstrated security monitoring using Splunk for a Visa Consultancy website.

The activity improved understanding of:

- Splunk installation and configuration
- Log collection and indexing
- SPL query creation
- Threat detection techniques
- Dashboard development
- Security monitoring
- Incident analysis

Continuous monitoring using SIEM tools such as Splunk is essential for detecting cyber threats, improving incident response, and maintaining secure web applications.